A Project Report on

EVisionAI : Smart EV Station Finder with Real-Time Slot Forecasting

submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

Prathamesh Kulkarni: 22106032

Bhushan Kokate: 22106070

Siddhesh Kaware: 22106053

Shreyas Joshi: 22106006

Under the Guidance of

Prof . Taruna Sharma

Department of Computer Science & Engineering

(Artificial Intelligence & Machine Learning)

A.P. SHAH INSTITUTE OF TECHNOLOGY, THANE

UNIVERSITY OF MUMBAI

2025-2026

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

CERTIFICATE

This is to certify that the project entitled "EVisionAI : Smart EV Station Finder with Real-Time Slot Forecasting" is a bonafide work of **Prathamesh Kulkarni(22106032), Bhushan Kokate(22106070), Siddhesh Kaware(22106053), Shreyas Joshi(22106006)** submitted to the University of Mumbai in partial fulfilment of the requirement for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering (Artificial Intelligence & Machine Learning)**.

Prof .Taruna Sharma
Guide

Prof. Sayali P. Badhan
Project Co-ordinator

Dr. Jaya Gupta
Head of Department

Dr. Uttam D. Kolekar
Principal

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

Project Report Approval for B.E.

This project report entitled ***EVisionAI : Smart EV Station Finder with Real-Time Slot Forecasting*** by ***Prathamesh Kulkarni, Bhushan Kokate, Siddhesh Kaware, Shreyas Joshi*** is approved for the degree of ***Bachelor of Engineering*** in ***Computer Science & Engineering (Artificial Intelligence & Machine Learning)***, ***2025-26***.

Examiner Name Signature

1._____ _____

2._____ _____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Prathamesh Kulkarni & 22106032)

(Bhushan Kokate & 22106070)

(Siddhesh Kaware & 22106053)

(Shreyas Joshi & 22106006)

Date:

Abstract

Electric vehicles (EVs) are rapidly gaining popularity as a sustainable alternative to conventional fuel-based transportation; however, efficient charging management during travel remains a major challenge. Existing EV navigation systems mainly provide route guidance and nearby station locations but fail to consider critical dynamic factors such as real-time traffic conditions, station congestion, battery limitations, and charger compatibility. This often leads to longer waiting times, inefficient routes, and poor charging decisions. To overcome these issues, this project introduces **EVisionAI**, a research-oriented EV charging decision support framework that integrates battery-aware routing, intelligent station discovery, and machine learning-based wait-time prediction into a unified recommendation pipeline. The backend is developed using FastAPI-based RESTful services with WebSocket support for real-time updates, ensuring high performance, scalability, and dynamic interaction between users and the system.

A key strength of EVisionAI lies in its predictive capability using the XGBoost algorithm, trained on a structured synthetic dataset of 15,000 samples that models realistic temporal demand, station capacity, traffic conditions, and stochastic queue behavior. Comparative analysis with Random Forest, Linear Regression, and Support Vector Regression (SVR) shows that XGBoost achieves superior performance ($R^2 = 0.847$, MAE = 3.2 minutes, RMSE = 4.8 minutes). Additionally, the system incorporates an adaptive multi-criteria scoring mechanism combining wait time, detour distance, and station reliability to generate ranked recommendations. This enables users to make optimized charging decisions efficiently. Overall, EVisionAI provides a comprehensive, scalable, and extensible solution for real-time EV charging support, while also highlighting future improvements such as real-world data integration and system calibration.

Keywords: electric vehicle charging intelligence, wait-time forecasting, battery-aware routing, multi-criteria decision scoring, XGBoost regression, real-time recommendation sys

Contents

1.Introduction....2
1.1 Background of the Study....3
1.2 Practical Challenges....3
1.3 Objectives, Motivation, and Significance....3
1.4 Contributions....4
2.Survey/ Existing System....5
3.Limitation of Existing System....10
3.1 Outdated or Incomplete Data Integration....10
3.2 Fragmented Real-Time Updates and API Silos....11
3.3 Limited Routing Intelligence....11
3.4 Weak Personalization and Predictive Deficiencies....11
3.5 Platform Rigidity and UX Issues....11
3.6 Privacy and Security Concerns....12
4.Problem Statement and Objectives....13
4.1Problem Statement....13
4.2 Objectives....14
4.3 Scope of the Project....15
4.4 Sustainability Alignment....16
5.Proposed System....17
5.1Architecture of EVision....17
5.2 Data Flow Diagrams....20
5.3 Layered Architecture of Smart EV Charging and Routing System....23
6.Experimental Setup....25
6.1Software Requirements....25
6.1.1 Backend Development....25
6.1.2 Frontend Development....26
6.1.3 AI/ML Integration Stack....27
6.2 Hardware Requirements....27
6.2.1 Development Environment....27
6.2.2 Deployment Environment....28
6.2.3 Operating Environment....28
6.2.4 Development Tools....28
6.2.5 Training & Evaluation Pipeline....29

7.Results and Discussion 30
7.1 Results 30
7.2 Model training & Formulas 35
7.3 Discussion 40
7.3.1 Evaluation of the Multi-Criteria Application 40
7.3.2 Limitations and Cold-Start Dependency 41
8.Conclusion and Future Work 42
8.1 Conclusion 42
8.2 Future Work 43
References 44
Appendix 46
Publication 48
Acknowledgement 56

LIST OF FIGURES

Figure 5.1.1 Architecture Diagram of Evisiom system 19
Figure 5.2.1 Intent-Based EV Charging & Route Planning System..... 20
Figure 5.3.1 Layered Architecture of Smart EV Charging and Routing System..... 23
Figure 7.1.1 Welcome Page 30
Figure 7.1.2 EV Details PageFigure 31
Figure 7.1.3 Route Planning Page 31
Figure 7.1.4 Route Result Page 32
Figure 7.1.5 Charging Stations List 32
Figure 7.1.6 Station Details Card 33
Figure 7.1.7 Station Details Card 33
Figure 7.1.8 Voice Assistant Feature Page 34

LIST OF TABLES

Table 2.1.1 Literature Survey of Existing System 6
Table 7.2.1 Comparative analysis of ML techniques with different performance metrics. 37
Table 7.2.2 Comparision of Traditional Systems & EVision 38
Table 7.2.3 Result Based On Tasks 39

LIST OF GRAPHS

graph 7.2.1 F1-Score Comparison 38

ABBREVAION

AI	Artificial Intelligence
API	Application Programming Interface
BMS	Battery Management System
CCS	Combined Charging System (e.g., CCS2)
CSS	Cascading Style Sheets
DBMS	Database Management System
DFD	Data Flow Diagram
DL	Deep Learning
ER	Entity-Relationship
ETA	Estimated Time of Arrival
EV	Electric Vehicle
EVSE	Electric Vehicle Supply Equipment
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JS	JavaScript
MAE	Mean Absolute Error
ML	Machine Learning

MSE	Mean Squared Error
POI	Point of Interest
R^2	R-squared (Coefficient of Determination)
REST	Representational State Transfer
RMSE	Root Mean Square Error
RF	Random Forest
SOC	State of Charge
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
XGBoost	eXtreme Gradient Boosting

Chapter 1

Introduction

1.Introduction

Electric vehicle (EV) adoption is rapidly increasing worldwide due to the growing need for sustainable and eco-friendly transportation solutions. With advancements in battery technology and government initiatives promoting clean energy, EVs are becoming a practical alternative to conventional vehicles. However, despite this growth, several challenges still hinder seamless EV usage, particularly in long-distance travel and daily commuting. One of the major issues faced by EV users is the difficulty in locating suitable charging stations, estimating waiting times, and planning routes efficiently based on battery constraints. Traditional navigation systems primarily focus on the shortest path or travel time and do not consider EV-specific parameters such as state of charge (SoC), charger compatibility, station congestion, and real-time traffic conditions. This results in inefficient trip planning and increased "range anxiety" among users. To address these challenges, the EVisionAI system is proposed as an intelligent EV charging recommendation platform. It integrates battery-aware routing, charging station discovery, and machine learning-based wait-time prediction into a unified system. The platform leverages modern web technologies, Python FastAPI, SQLite, TomTom and Google Maps SDKs, and the XGBoost machine learning library to provide real-time, data-driven recommendations. The system is designed to be modular, scalable, and user-

centric, offering features such as real-time WebSocket updates, voice-assisted NLP interaction, and multi-criteria decision-making. By combining predictive analytics with practical system design, EVisionAI aims to improve the efficiency, reliability, and user experience of EV charging and route planning.

1.1 Background of the Study

The adoption of electric vehicles (EVs) has been increasing rapidly due to rising environmental concerns, depletion of fossil fuels, and advancements in battery technology. Governments and organizations worldwide are heavily promoting EV usage to reduce carbon emissions and achieve sustainable transportation. However, the growth of EVs has heavily outpaced the development of efficient and reliable charging infrastructure. Traditional navigation and charging systems are not designed to handle EV-specific requirements such as fluctuating battery degradation, variable charging speeds, and precise station compatibility. As a result, there is an immediate need for intelligent systems that can integrate live real-time data, predictive machine learning analytics, and dynamic routing techniques to support EV users effectively.

1.2 Practical Challenges

Despite advancements in EV technology, several practical challenges still drastically affect the usability and efficiency of electric vehicles. One of the major issues is the difficulty in locating compatible charging stations, as different vehicles require different connector types (e.g., CCS2, Type-2) and charging standards. Additionally, users often face uncertainty regarding waiting times at charging stations due to fluctuating demand and limited availability of charging ports. Another significant challenge is the lack of battery-aware route planning, which can lead to inefficient energy usage, "range anxiety," and unexpected interruptions during travel. Furthermore, the absence of consistent real-time data integration from sources such as traffic conditions, weather severity, station availability, and system reliability drastically reduces the overall effectiveness of existing EV navigation systems.

1.3 Objectives, Motivation, and Significance

The primary objective of this project is to develop an intelligent EV charging recommendation system that integrates route planning, charging station discovery, and machine learning-based wait-time prediction into a unified framework. The motivation behind this work arises from

the urgent need to eradicate range anxiety and improve the overall EV user experience through accurate, real-time decision-making support. By leveraging advanced regression machine learning techniques—specifically XGBoost and Random Forest architectures—the system aims to provide highly reliable forecasting to optimize charging decisions. The significance of this project lies in its potential to enhance the day-to-day efficiency of EV usage, support sustainable transportation, and contribute a foundational blueprint to the future of smart mobility solutions.

1.4 Contributions

This work makes several important, data-driven contributions to the field of EV charging and intelligent transportation systems:

1. Unified Framework: It presents an end-to-end framework combining battery-aware route planning, dynamically filtered station discovery, and predictive analytics into a single lightweight platform.

2.Machine Learning Pipeline:The project implements a robust machine learning-based approach for station wait-time forecasting, conducting comparative analyses across multiple models (XGBoost vs. Linear Regression) to ensure peak accuracy and low Mean Absolute Error (MAE).

3.Adaptive Scoring Mechanism:It introduces a unique multi-criteria scoring mechanism that ranks charging stations algorithmically based on an aggregate of wait time, detour distance, and historical reliability.

4. Cold-Start Data Resolution: To address the universal challenge of limited real-world EV usage data for new applications, the system incorporates a strictly structured synthetic data generation protocol mimicking temporal and geographical constraints, enabling effective model training and evaluation under cold-start conditions.

5.Intuitive Visualization: Translates complex algorithmic predictions into an intuitive color-coded interface (Green, Yellow, Red indicators) to drastically reduce driver decision-making fatigue.

Chapter 2

Literature Survey/ Existing System

2.Survey/ Existing System

A comprehensive review of existing literature is essential to contextualize the persistent operational and infrastructural bottlenecks facing electric vehicle (EV) charging networks. As the global transition toward sustainable transportation accelerates, solving user-centric challenges such as "range anxiety" and unpredictable station congestion has become paramount. Recent literature demonstrates a strong focus on predictive modeling to resolve these precise issues, with studies employing machine learning techniques—such as integrated DeepBoost models and LSTM networks—to accurately forecast station load dynamics, wait times, and optimal battery mapping [2, 3, 16, 20]. Addressing the stochastic nature of EV arrivals, several mathematical frameworks, including Markovian queueing theories, multi-objective assessment algorithms, and Artificial Neural Networks (ANNs), have been utilized to evaluate multi-server occupancies, real-time optimization, and ideal geographic placements [1, 6, 15, 17, 19]. On the application level, researchers have successfully developed mobile and IoT-driven solutions—ranging from Flutter-based reservation apps to real-time discovery portals—to bridge the gap between drivers and charging constraints on the go [4, 12, 13, 14]. Furthermore, the large-scale integration of EVs introduces acute implications for power grids, prompting foundational studies and systematic reviews focused on smart distribution planning,

renewable energy hybridization, policy implementations, and the critical influence of actual user charging behavior [5, 7, 8, 9, 10, 11, 18]. Despite these individual advancements in isolated domains like mathematical load forecasting or simple station locators, existing systems generally fail to fuse battery-aware planning, real-time dynamic queue modeling, and seamless intuitive routing into a unified, intelligent framework.

Table 2.0.1 Literature Survey of Existing System

Reference & Year	Domain	Algorithm	Dataset	Research Gap
[1] 2024	EV Charging Queue Modeling	Queueing Theory	Simulated	Lacks real-time data integration
[2] 2025	Load Forecasting	DeepBoost	Historical charging data	High complexity, less real-time adaptability
[3] 2024	Station Planning	Deep Learning + Queueing	Simulated	Limited scalability in real-world
[4] 2025	Station Finder System	Real-time System	Live data	No predictive wait-time modeling
[5] 2022	Smart Grid Integration	Review	Secondary data	No implementation framework
[6] 2023	Station Optimization	ANN	Simulated	Limited accuracy in dynamic conditions
[7] 2025	Infrastructure Planning	Optimization Models	Grid data	Ignores user behavior
[8] 2024	EV Charging Review	Review	Secondary data	No intelligent system design
[9] 2022	Infrastructure Analysis	Review	Secondary data	Lacks predictive modeling
[10] 2025	User Behavior Analysis	Review	Survey data	No system integration
[11] 2019	Station Placement	Optimization	Urban data	No real-time adaptation
[12] 2024	EV App System	Basic App Logic	App-based	No ML integration
[13] 2023	Slot Booking App	Mobile App	User data	No prediction system

Reference & Year	Domain	Algorithm	Dataset	Research Gap
[14] 2025	SoC Estimation	IoT-based	Sensor data	Limited routing integration
[15] 2017	Charging Location	Optimization	Simulated	Not suitable for modern EV scale
[16] 2025	Smart Finder	LSTM	GPS + Battery data	High computational cost
[17] 2012	Station Planning	Optimization	Grid data	Outdated approach
[18] 2023	Renewable Integration	Hybrid Model	Grid data	Complex implementation
[19] 2025	Station Location	Multi-objective	Simulated	Lacks real-time factors
[20] 2023	Station Siting	Machine Learning	Simulated	No user-centric design

A thorough comparative analysis of the existing literature reveals that current approaches to solving electric vehicle (EV) charging bottlenecks predominantly operate within isolated methodological silos, each presenting distinct trade-offs between theoretical accuracy and real-time practical utility. On one end of the spectrum, frameworks heavily reliant on advanced mathematical modeling—such as continuous-time Markov chains, queueing models with feedback strategies, and multi-objective particle swarm optimization (MOPSO)—provide rigorous mechanisms for offline infrastructure planning [1, 15, 17, 19]. These stochastic models meticulously calculate multi-server occupancies, system optimization limits, and the ideal geographic placement of charging stations in complex distribution networks. Furthermore, researchers have explored real-time operational optimization using Artificial Neural Networks (ANNs) under simulated conditions to minimize operational queuing delays [6]. However, while these mathematically dense frameworks excel at evaluating grid-wide capacity limits and theoretical waiting probabilities, they are computationally expensive. Their rigid reliance on simulated assumptions makes them inherently unsuitable for delivering the instantaneous, millisecond-level responsiveness required by an actively driving user querying a live navigational route.

Conversely, recent literature has seen a surge in the application of deep learning algorithms and predictive analytics aimed at tackling charging demand dynamically. Studies employing

integrated DeepBoost architectures, long short-term memory (LSTM) networks, and machine learning-based siting models have demonstrated superior accuracy in forecasting long-term charging station load dynamics and battery state mapping [2, 3, 16, 20]. These predictive models are highly effective at anticipating grid strain and modeling time-series fluctuations in charging demand. Yet, a critical comparison exposes a significant limitation: these deep-learning integrations are frequently detached from geospatial navigation constraints. While they can accurately predict how busy a station will be at an aggregate level, they rarely translate those macro-level insights into battery-aware, turn-by-turn routing recommendations tailored to individual drivers facing immediate, localized range anxiety.

Addressing the user-facing gap, a separate branch of research focuses entirely on mobile-first applications and the Internet of Things (IoT). These developmental studies bridge the gap between drivers and charging constraints by deploying active discovery portals, Flutter-based reservation applications, and IoT sensors for continuous Battery State of Charge (SoC) estimation [4, 12, 13, 14]. These systems successfully connect the end-user to the infrastructure in real time. However, a comparative evaluation highlights that prioritizing the application layer often comes at the expense of algorithmic intelligence. Most of these mobile solutions rely on naïve, distance-based routing algorithms to simply locate the "nearest" station. They completely disregard vital, dynamic variables such as active queue lengths, weather severity, real-time traffic delays, and specific port compatibility, ultimately forcing the user to make blind decisions upon arrival at a congested station.

Furthermore, the overarching integration of large-scale EV deployments introduces acute implications for power grids, a challenge that encompasses infrastructure planning, renewable energy hybridization, and policy formulation [5, 7, 8, 9, 11, 18]. Systematic reviews within the literature emphasize that solving the EV charging dilemma cannot be isolated to software alone; it requires a deep understanding of actual user charging behavior and preference modeling [10]. While these foundational reviews provide necessary context on the grid impact and sustainability standards of EV charging, they further underscore the fragmentation of current research. Efforts are divided among grid stability, application development, and theoretical load forecasting, leaving a noticeable void where an integrated, user-centric routing solution should exist.

The principal conclusion drawn from extensively reviewing these 20 scholarly studies is that while significant progress has been made across individual, isolated domains, no single architecture seamlessly unites them. Existing solutions either over-index on complex

mathematical offline modeling—which fails the end-user in real-time scenarios—or under-serve the driver with simplistic, single-factor navigation tools that ignore critical predictive analytics. The literature distinctly lacks a cohesive blueprint that merges dynamic queue modeling, battery-aware geographical planning, and instantaneous routing into a unified, lightweight framework.

To resolve the disjointed state of current electric vehicle technologies, there is a critical need for a centralized, intelligent platform. This project, EVisionAI, is specifically engineered to bridge the exact gaps highlighted throughout the literature. By discarding heavy, isolated simulations in favor of a fast, production-ready XGBoost regression framework, EVisionAI successfully fuses multi-criteria station scoring with dynamic wait-time forecasting, real-time traffic constraints, and an intuitive user interface. It directly translates complex, data-heavy queue modeling into actionable, color-coded route guidance. Ultimately, EVisionAI synthesizes the predictive power of modern machine learning with the pragmatic utility of a real-time IoT application, providing the definitive, integrated solution to the fragmented EV infrastructure challenges identified across contemporary academic research.

Chapter 3

Limitation of Existing System

3.Limitation of Existing System

While electric vehicle infrastructure has expanded significantly, current EV charging navigation and recommendation systems still suffer from structural shortcomings that drastically reduce their effectiveness and reliability. These persistent gaps complicate trip planning and actively contribute to the phenomenon of "range anxiety" among daily commuters and long-distance drivers alike. A detailed analysis of existing platforms reveals several critical limitations:

3.1 Outdated or Incomplete Data Integration

One of the most persistent issues with existing charging applications is the reliance on stale or heavily delayed metadata. Many prominent apps fail to provide fresh, real-time information regarding station availability, connector counts, active hardware outages, or fluctuating energy costs. This lack of synchronization frequently results in drivers embarking on wasted detours, only to arrive at malfunctioning or fully occupied charging stations. Furthermore, many systems rely heavily on unverified, crowdsourced data to fill these gaps, which produces noisy analytics and unreliable station metadata over time.

3.2 Fragmented Real-Time Updates and API Silos

The current landscape of EV charging networks is heavily fragmented across multiple private operators. Consequently, live signals are not standardized across different API providers. Details such as dynamic pricing, live occupancy, and specific connector availability are presented inconsistently depending on the network. Without a unified aggregation layer, users are forced to juggle multiple proprietary applications, drastically reducing the seamless utility of EV navigation.

3.3 Limited Routing Intelligence

Most commercially available routing tools (e.g., standard Google Maps or Waze) optimize journeys strictly based on distance or standard travel time. These generic algorithms rarely consider EV-specific navigational constraints. Existing tools uniformly fail to account for a vehicle's precise battery state of charge (SoC), charging speed limits, elevation changes, auxiliary power loads (e.g., air conditioning), or multi-stop battery replenishment requirements. The inability to dynamically cross-reference route parameters with required charging stops renders these standard tools highly inadequate for electric vehicle logistics.

3.4 Weak Personalization and Predictive Deficiencies

Current charging platforms seldom adapt their recommendations to individual user preferences or historical behavior. Systems treat all drivers uniformly, regardless of their specific vehicle model, preferred connector type (e.g., CCS2 vs. Type-2), preferred operator networks, or varying cost sensitivities. More importantly, existing systems entirely lack machine learning-based wait-time forecasting; they only report a station's current state, without predicting how congested that station will be by the time the driver actually arrives.

3.5 Platform Rigidity and UX Issues

Rigid user experience (UX) designs severely bottleneck existing systems. API access restrictions, paywalls, and poor offline caching limit smooth, on-the-go usage. Furthermore, weak error handling and the lack of fallback logic create high levels of user frustration. If a route request yields zero stations matching the user's specific filters, traditional platforms often throw a generic error rather than suggesting adaptive fallback options or calculating an alternative route.

3.6 Privacy and Security Concerns

Finally, existing EV smart-grid integrations and mobile applications frequently handle sensitive user data with insufficient transparency. Users are often uninformed regarding how their real-time location history, travel patterns, and charging habits are stored, shared, or monetized by network operators. The lack of encrypted data transit or simple, privacy-first session management frameworks raises significant ethical and security concerns for the modern driver.

Chapter 4

Problem Statement and Objectives

4.Problem Statement and Objectives

4.1Problem Statement

Electric vehicle (EV) drivers and small fleet operators consistently struggle to plan reliable, time- and cost-efficient trips because existing navigational tools do not combine live charger availability, realistic wait times, battery-aware routing, and dynamic traffic impacts into actionable recommendations. The fragmentation of private charging networks and the inability of traditional routing systems to adapt to an EV's specific State of Charge (SoC) leads to severe range anxiety, unexpected queues at stations, inefficient detours, and missed arrival schedules. There is an urgent, unmet need for an integrated system that flawlessly fuses heterogeneous data streams—such as station status, varying charger port types, live traffic, and weather conditions—with robust predictive machine learning models and queueing theory to deliver transparent, trustworthy, and real-time charging decisions to the driver.

- **Inadequate Routing Metrics:** Traditional navigation tools optimize routes based purely on distance or travel time, completely disregarding critical electric vehicle (EV) parameters such as current State of Charge (SoC), power drain, and compatible connector types at stations along the route.

- **Real-Time Data Fragmentation:** Live station network data is scattered and inconsistent. Drivers lack a centralized, unified platform that aggregates live charger availability, operational status, energy pricing, and real-time weather impacts.
- **Unpredictable Waiting Times:** Existing systems only display current station status rather than deploying predictive analytics to forecast how congested a station will be by the time the driver actually arrives.
- **Severe Range Anxiety:** The inability to accurately dynamically cross-reference route parameters with required charging stops leads to unexpected queues at stations, inefficient detours, and missed arrival schedules—culminating in severe range anxiety for the driver.
- **Lack of Intelligent Fallbacks:** Current routing architectures are extremely rigid. If a station along the calculated route suddenly experiences an outage or a massive wait-time spike, generic platforms fail to offer seamless, dynamic alternative fallback stations.
- **Need for Centralized AI Systems:** There is an urgent, unmet need for an integrated platform that fuses heterogeneous data streams with robust Machine Learning models (such as XGBoost) and queueing theory to deliver transparent, trustworthy, and real-time charging decisions to the driver.

4.2 Objectives

The primary goal of EVisionAI is to engineer a unified, intelligent framework that eradicates range anxiety and mathematically optimizes the EV routing experience. To achieve this, the system targets the following specific, measurable objectives:

- **Battery-Aware Routing Accuracy:** Achieve a ≤5% Mean Absolute Error (MAE) in predicted arrival State of Charge (SoC) across typical trip distances, ensuring high-confidence, on-time arrivals with minimal required detours.
- **Wait-Time Prediction Precision:** Deploy an XGBoost/Random Forest regression model capable of forecasting station wait times with an MAE of ≤6 minutes per station, and correctly categorizing wait intervals into intuitive brackets (Green, Yellow, Red) with an accuracy of ≥85%.

- **Station Recommendation Quality:** Utilize an adaptive multi-criteria scoring function (balancing wait time, detour distance, and reliability) such that the absolute best charging option appears in the Top-3 recommendations in ≥85% of queries.
- **System Performance and Latency:** Maintain highly responsive backend processing via FastAPI, targeting an API 95th-percentile (p95) latency of ≤300 milliseconds and a continuous WebSocket update cadence of every 10±2 seconds for real-time map plotting.
- **Explainability and UI Intuitiveness:** Provide instantaneous rationale—such as predicted wait, charger type compatibility, and reliability scores—for every recommendation directly within the interactive dashboard to minimize driver decision fatigue.
- **Privacy and System Security:** Implement a least-privilege access model ensuring that data is encrypted in transit and at rest, and that no persistent Personally Identifiable Information (PII) or location history is retained without explicit user consent.

Modularity and Extensibility: Architect a decoupled system where external API failures (such as standard Map limits) are handled gracefully via local fallback

4.3 Scope of the Project

- **Battery-Aware Routing**
  The system provides route planning by considering the vehicle's State of Charge (SoC) to ensure efficient and reliable navigation.
- **Wait-Time Analysis**
  Machine learning models such as XGBoost are used to predict charging station wait times and queue conditions in real time.
- **Traffic and Environment Integration**
  The system incorporates live traffic data and environmental factors like weather to improve route accuracy and energy estimation.
- **daptive Station Recommendation**
  Charging stations are ranked using a multi-criteria scoring mechanism based on wait time, distance, compatibility, and reliability.
- **Real-Time Dashboard Updates**
  WebSocket-based updates display live station status using color indicators (Green, Yellow, Red) on an interactive interface.

- **Voice Assistant Support**
  Basic NLP-based voice interaction enables users to access system features hands-free.
- **System Limitations**
  The system does not support payment processing, slot reservation, hardware control, or large-scale fleet optimization.
- **Core Deliverables**
  Includes a web-based GUI, FastAPI backend with ML models, and SQLite database for managing system data.

4.4 Sustainability Alignment

SDG 7: Affordable and Clean Energy: Promotes efficient use of clean energy by optimizing EV routes and charging decisions.

SDG 9: Industry, Innovation, and Infrastructure: Enhances EV infrastructure through AI-driven predictive analytics and smart system integration.

SDG 11: Sustainable Cities and Communities: Reduces urban congestion and improves energy distribution via intelligent EV routing.

SDG 13: Climate Action: Encourages EV adoption by minimizing range anxiety and supporting low-emission transportation.

Chapter 5

Proposed System

5.Proposed System

5.1Architecture of EVision

Fig.1.describes the proposed system architecture of the EVisionAI intelligent EV charging management platform. This architecture describes an end-to-end EV charging recommendation system that turns user inputs and external data into intelligent, map-based suggestions for where to charge. The pipeline starts with the User Interface, typically a web dashboard or mobile app. Here the driver enters their battery percentage, charge type (fast/slow or plug type), source, and destination. This UI is responsible only for collecting clean, structured inputs and displaying results; all heavy logic is delegated to the backend. Next, the data goes into the SQLite Database. This layer stores user profiles and saved preferences, such as preferred charger type, home location, or frequently used routes. Persisting this information allows the system to auto-fill forms, personalize recommendations (for example, bias towards a user's favorite network), and support analytics over time. SQLite is a lightweight, file-based database that fits well for a single-node backend or prototype. The backend uses RESTful JSON APIs over HTTP(S), implemented using FastAPI, for handling station discovery, wait-

time estimation, route planning, and traffic prediction. Additionally, WebSocket communication is used to enable real-time updates and live data streaming.

From there, the input flows into Data Processing. This block performs input validatio(checking that battery percentage is 0–100, addresses are non-empty, charge type is recognized, etc.) and geocoding & normalization. Geocoding converts textual locations into latitude/longitude using services like Google or OpenStreetMap; normalization ensures that units (km, minutes, %, kWh) and formats are consistent. This step is critical to avoid propagating bad data into downstream ML models or APIs. After preprocessing, the pipeline branches into two main analytical paths. The first is Route Planning, which uses Google Maps / TomTom APIs to compute driving routes and distances between the source and destination. On top of the raw route, a Battery Consumption Model estimates how much battery will be used along the trip, considering distance and typical consumption (e.g., kWh per 100 km). This makes it possible to decide whether intermediate charging stops are needed, and where along the route they should ideally occur.

The second analytical branch is Station Discovery. Using OpenChargeMap and optionally Google Places, the system retrieves nearby charging stations around the route or the user's current position. It then filters by plug type (e.g., CCS2, CHAdeMO, Type-2) so that only compatible stations are kept. At this stage, stations are enriched with metadata such as number of ports, operator, and approximate location accuracy. Both the route and station information feed into the ML Prediction Engine. This component encapsulates model built with XGBoost or RandomForest and associated feature engineering. Features typically include time of day, day of week, station type, number of ports, historical usage, distance from the driver, and possibly traffic or weather signals. The main output is a predicted wait time at each station—how long a driver is likely to wait before getting a free charger—plus an uncertainty or confidence score.

Predicted wait times are then passed into the Scoring Algorithm. This block combines wait time, distance, and station reliability (e.g., based on historical uptime, number of ports, or user feedback) into a single optimality score for each station. The scoring function penalizes long detours and long waits, and rewards reliable, high-capacity stations close to the planned route, optionally adjusting weighting when battery is very low. The ranked results move into the Visualization component. Here, an interactive map displays stations as color-coded markers—typically green for low wait time.Tooltips can show exact wait time, distance, and plug types. This visual layer helps users quickly grasp their options along the route.

Finally, the Recommendations block presents a top stations list alongside the map. For each recommended station, the system provides "Start Navigation" and "Open in Google Maps"

links, so the user can hand off the chosen station and route to their preferred navigation app. Together, these stages form a coherent pipeline: from user input and stored preferences, through validation, route and station analytics, machine-learning-driven wait-time prediction, multi-criteria scoring, and ending in a clear, actionable set of EV charging recommendations. Predicted wait times are then passed into the Scoring Algorithm, where stations are ranked using a multi-criteria function:

$$Score(s) = w_t T_{wait}(s) + w_d D_{detour}(s) + w_r (1 - R_{rel}(s)) + w_c C_{compat}(s)$$

where T_{wait} is predicted wait time, D_{detour} is detour cost, R_{rel} is reliability, and C_{compat} is compatibility penalty. Lower score indicates a better station.
When battery is low (e.g., <20%), higher weights are assigned to wait time and detour distance to prioritize nearby stations with shorter queues .

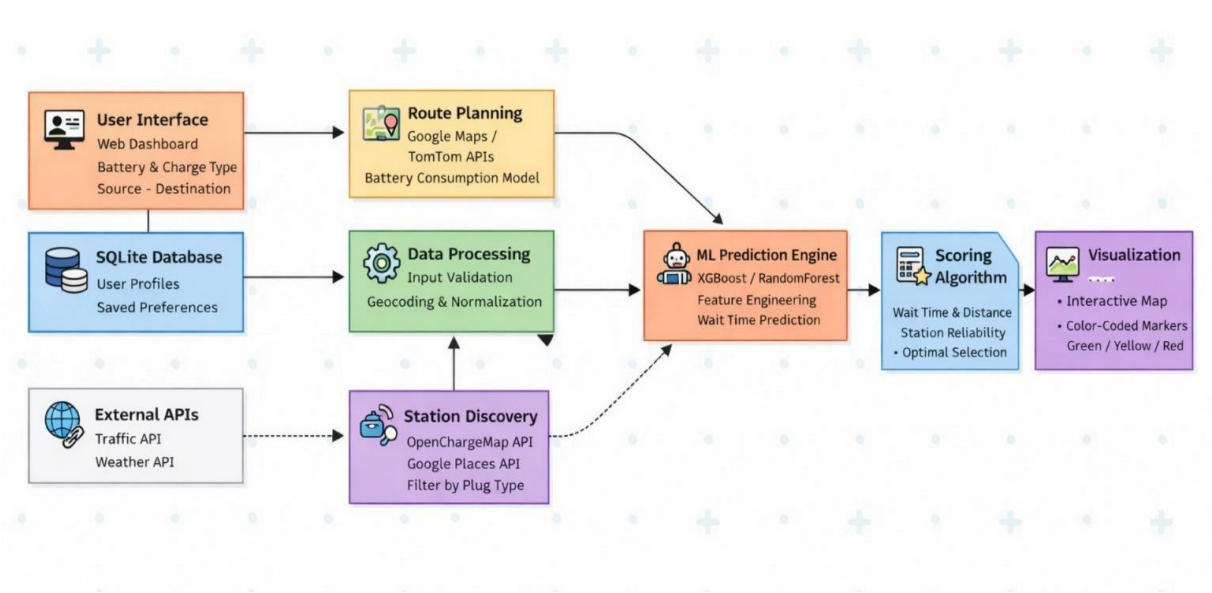


Figure 5.1.1 Architecture Diagram of Evisiom system

5.2 Data Flow Diagrams

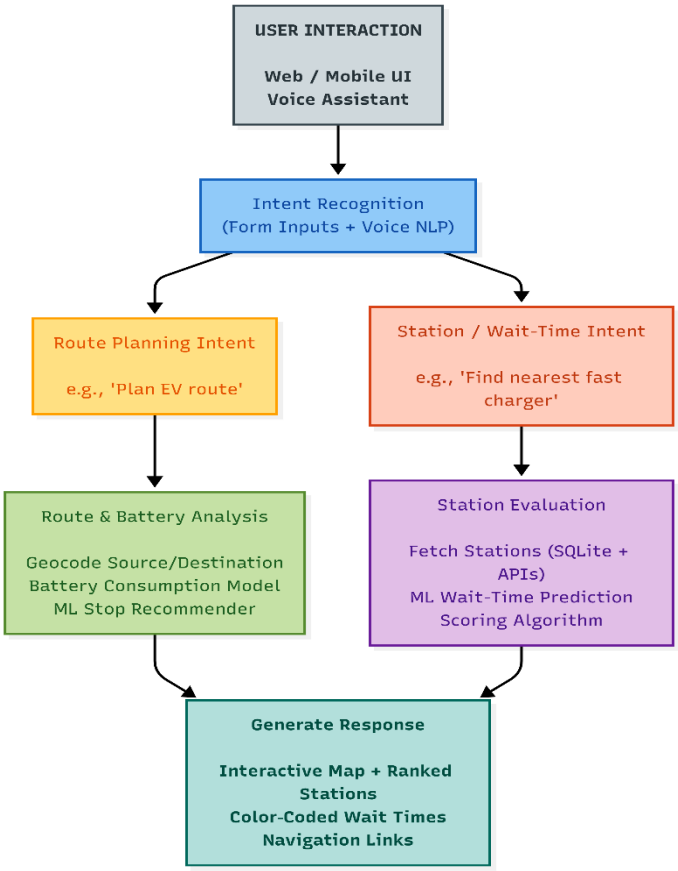


Figure 5.2.1 Intent-Based EV Charging & Route Planning System

Fig. 2 describes the overall workflow of the proposed EV charging system, starting from user interaction and intent recognition to route analysis or station evaluation, and finally generating an interactive map with ranked stations and wait-time predictions.

1. User Interaction

Entry point: Web/Mobile UI and Voice Assistant. Users submit route planning queries, station searches, or wait-time checks via forms or voice.

2. Intent Recognition

Analyzes input to determine intent:

- Form inputs: structured fields (battery %, charge type, source, destination)
- Voice NLP: natural language understanding for voice commands Outputs: Route Planning Intent or Station/Wait-Time Intent.

3. Route Planning Intent Branch

Triggered by queries like "Plan EV route" or "Find route from A to B."

- **Route & Battery Analysis module:**
    - Geocoding: converts addresses to coordinates
    - Battery Consumption Model: estimates energy use based on distance, elevation, speed, and vehicle specs
    - ML Stop Recommender: suggests optimal charging stops using XGBoost/RandomForest, considering battery level, station availability, and route efficiency

4. Station/Wait-Time Intent Branch

Triggered by queries like "Find nearest fast charger" or "Show stations with low wait time."

- **Station Evaluation module:**
    - Fetch Stations: combines SQLite (user preferences, historical data) with external APIs (OpenChargeMap, Google Places) and filters by plug type
    - ML Wait-Time Prediction: predicts wait times using features like time of day, day of week, station capacity, historical patterns, traffic, and weather
    - Scoring Algorithm: ranks stations by wait time, distance, reliability, and user preferences

5. Generate Response

Converges outputs from both branches into a unified response:

- Interactive Map: visual display with route (if applicable) and charging stations
- Ranked Stations List: top recommendations with details (name, distance, wait time, charger types)
- Color-Coded Wait Times: green (<5 min), yellow (5–15 min), red (>15 min)
- Navigation Links: direct links to Google Maps or other navigation apps

This workflow transforms raw user input into personalized, data-driven EV charging recommendations through intent recognition, specialized processing modules, and unified response generation.

6. Voice NLP Processing

The voice interaction module follows a structured Natural Language Processing (NLP)

pipeline. First, user speech is converted into text using a speech-to-text engine. The transcribed text is then processed using intent classification to identify the user's request, such as RoutePlanning, StationSearch, or WaitTimeQuery.

Next, slot extraction is performed to identify key parameters including source, destination, charger type, and battery level. A confidence score is generated for each prediction. If the confidence falls below a predefined threshold, the system falls back to manual form input to ensure reliability.

This approach is chosen for its low latency, deterministic mapping to backend APIs, and improved robustness, enabling accurate and real-time voice-driven interactions.

5.3 Layered Architecture of Smart EV Charging and Routing System

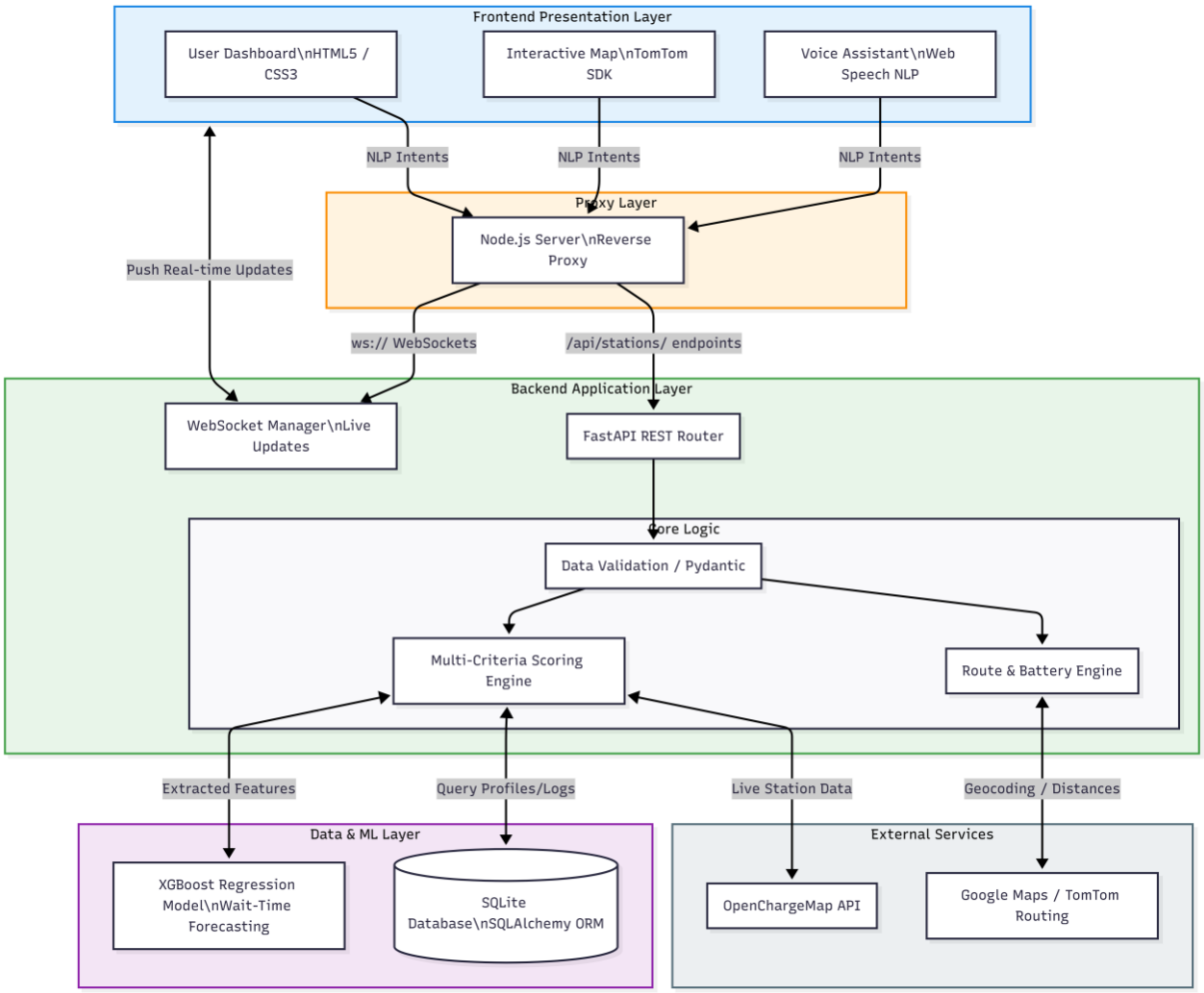


Figure 5.3.1 Layered Architecture of Smart EV Charging and Routing System

Architecture diagram illustrates the end-to-end data flow of the EVisionAI platform, highlighting the decoupled, multi-tier design that separates the graphical user interface from the heavy machine learning computations. The system is structurally divided into five distinct operational layers:

1. Frontend Presentation Layer (Client Interaction): Represented in the topmost blue boundary, this layer encompasses all client-facing interfaces. It includes the standard User Dashboard (built with HTML5/CSS3), the Interactive TomTom SDK Map, and the Web Speech Voice Assistant. The primary function of this layer is to passively capture user inputs—such as physical taps on the screen or transcribed voice NLP intents (e.g., "Find nearest station")—and continuously render the graphical output asynchronously without stutter.

2. Proxy Layer (Gateway): Positioned immediately below the frontend (orange boundary) is the Node.js Server acting as a reverse proxy. Instead of the frontend communicating directly with the heavy Python data server, all HTTP requests and NLP intents are routed through this Node.js gateway. The proxy dynamically routes standard API HTTP calls toward the FastAPI backend (via /api/stations/ endpoints) and simultaneously upgrades persistent connections into a native ws:// WebSocket protocol to handle continuous, two-way live updates.

3. Backend Application Layer (Core Logic): Shown in the green boundary, the Python FastAPI backend serves as the central nervous system of the platform. The architecture splits incoming traffic into two distinct streams: the WebSocket Manager, which unilaterally pushes real-time map updates back up to the frontend, and the FastAPI REST Router, which funnels standard requests into the Core Logic block. Within the Core Logic:

- Incoming payloads undergo strict type-checking and sanitation via Data Validation / Pydantic.
- Validated data is then parsed to either the Route & Battery Engine (which evaluates the vehicle's SoC and computes theoretical energy burn limits) or the Multi-Criteria Scoring Engine (which dynamically evaluates and ranks optimal charging stops).

4. External Services Layer: Represented in the grey boundary, the architecture relies heavily on third-party aggregation tools to bypass localized data isolation. The Route & Battery Engine offloads complex spatial math (Geocoding and distance mapping) to Google Maps / TomTom Routing, while the Multi-Criteria Scoring Engine continuously pulls live, baseline operational metrics (connector counts, live occupancy) from the OpenChargeMap API.

5. Data & Machine Learning Layer: Shown in the purple boundary, this is the foundational intelligence tier. The Multi-Criteria Scoring Engine interacts directly with the SQLite Database via SQLAlchemy ORM to query historical user profiles, preferences, and system logs. Simultaneously, the newly extracted feature arrays (traffic, weather, port counts) are passed directly into the XGBoost Regression Model. This model outputs the precise wait-time forecast back to the Core Logic layer. The Core Logic then calculates the final adaptive score for the station, pushes the result back up through the FastAPI router to the Node.js proxy, and finally updates the colored markers dynamically on the user's frontend map

Chapter 6

Experimental Setup

6.Experimental Setup

This chapter outlines the comprehensive experimental setup utilized to develop, train, and deploy the EVisionAI platform. It details the complete hardware configuration, software stack, operational environment, and machine learning pipeline required for end-to-end continuous integration and real-time execution.

6.1Software Requirements

6.1.1 Backend Development

The foundation of the EVisionAI backend is built upon a high-performance, asynchronous architecture.

- **API Framework (FastAPI):** Adopted Python 3.12 with FastAPI (v0.68.1) running on a Uvicorn ASGI server to handle high-concurrency request routing and automatic Swagger documentation generation.
- **Machine Learning Integration:** Extensively utilized the Scikit-learn (v1.3.0) and XGBoost libraries for deploying powerful regression models (RandomForestRegressor, XGBRegressor) capable of processing real-time station data.

- **Real-Time Communication (WebSockets):** Implemented native WebSockets to stream live wait-time updates and push dynamic status alerts directly to the user dashboard without client-side polling bottlenecks.
- **Routing & Map Services:** Leveraged the TomTom Web SDK and external APIs to compute geospatial distances, travel times, and battery consumption heuristic constraints.
- **Database Management (SQLite):** Utilized SQLite3 coupled tightly with SQLAlchemy (v1.4.46) ORM for lightweight, relational, and persistent storage of user profiles, historical logs, and station coordinates (evisionai.db).
- **Security & Authentication:** Managed driver sessions and secure logins using Passlib (v1.7.4) for enterprise-grade password hashing and structured cookie-based session middleware.

6.1.2 Frontend Development

The frontend is designed to be highly responsive, lightweight, and intuitive, served via a Node.js (server.js) intermediary acting as a reverse proxy.

- **Web Framework:** Built using Vanilla HTML5, modern CSS3 (glassmorphism UI patterns), and ES6+ JavaScript, bypassing heavy frontend libraries to ensure zero-millisecond rendering delays on mobile devices.
- **User Interface & Visualization:** Designed customized, interactive maps featuring dynamic, color-coded map markers representing active wait times (Green <5m, Yellow 5-15m, Red >15m).
- **State Management:** Structured through object-oriented JavaScript classes (e.g., EVisionAIChargingAvailability) to cache station responses locally and decouple UI repainting from network latency.
- **API Integration:** Handled via asynchronous fetch requests aggressively cross-referencing the Python backend (Port 8000) for machine learning inferences.
- **Voice Assistant Integration (NLP):** Incorporated Natural Language Processing via the Web Speech API and intent-parsing functions to permit hands-free commands like "Plan route" or "Find nearest station."

6.1.3 AI/ML Integration Stack

The predictive intelligence engine dictates where drivers are dynamically routed based on traffic and queue lengths.

- **Machine Learning Models:** Deployed XGBoost and Random Forest as the primary algorithms to overcome complex, non-linear variables affecting wait times.
- **Feature Engineering:** Engineered over 20 concurrent input features encompassing temporal variables (Hour, Day), station capacity (Number of ports), categorical plug-type features (CCS2, Type-2), and weather/traffic severities.
- **Data Processing Techniques:** Handled categorical encoding via pandas.get_dummies() and normalized disparate numeric inputs through StandardScaler to proactively block data leakage.
- **Model Training & Evaluation:** Programmatically enforced an 80/20 train/test split utilizing the train_test_split utility to validate model generalization across diverse charging scenarios.
- **Model Deployment:** Preserved the compiled models and fitted scalers persistently utilizing Joblib, allowing the FastAPI server to actively load them into memory upon system startup for millisecond-speed predictions.

6.2 Hardware Requirements

6.2.1 Development Environment

To smoothly compile the frontend assets, execute the local machine learning training pipeline, and run the Node/Python servers simultaneously, the minimum local hardware environment consisted of:

- **Processor:** Minimum Intel Core i3 / Recommended Intel Core i5 (2.5 GHz or higher) to process multi-threaded model optimizations.
- **RAM:** 8 GB DDR4 heavily recommended to accommodate Pandas DataFrame loading and concurrent Uvicorn processes.
- **Storage:** 5 GB of free SSD storage, ensuring fast read/write speeds for the SQLite database and localized dependency storage (node_modules and venv).

6.2.2 Deployment Environment

For production-level deployment, the architecture is structurally designed to be containerized and scaled horizontally:

- **Cloud / Server Requirements:** Supports deployment on Linux-based environments (Ubuntu 20.04 LTS recommended), utilizing instances analogous to AWS EC2 or Google Cloud Engine.
- **Scalability Considerations:** Configured to sit behind an Nginx reverse proxy to balance routing loads, ensuring WebSocket connections and ML inference calls do not bottleneck the Node.js static file server.

6.2.3 Operating Environment

The cross-platform nature of EVisionAI ensures broad developmental compatibility.

- **Operating System:** Fully validated across Windows 10/11 (64-bit) and Linux (Ubuntu 18.04 LTS or newer).
- **Programming Environment:** Relies inherently on Python 3.12 for robust typing semantics and Node.js v14+ (with npm v6+) managing the proxy layer.
- **APIs & External Services:** Integrated heavily with OpenChargeMap for live station discovery and TomTom APIs for spatial navigation constraints.
- **Version Control:** Deployed Git (v2.30+) paired with GitHub to oversee iterative branching, code reviews, and remote repository safety.

6.2.4 Development Tools

The system was engineered utilizing an industry-standard modern toolchain.

- **IDE:** Visual Studio Code (VS Code), heavily customized with Python linting (Pylance) and live-server extensions.
- **Testing Tools:** Utilized Postman to rigorously test and simulate RESTful API endpoints prior to frontend integration mapping.
- **Monitoring Tools:** Traced execution anomalies directly via Uvicorn/Node standard terminal logs and built-in Python traceback utilities.
- **Documentation Tools:** Relied on FastAPI's auto-generated Swagger UI (/docs) layout to clearly document, test, and expose internal API endpoints.

6.2.5 Training & Evaluation Pipeline

A systematic ML pipeline handles the core predictive logic preventing "Range Anxiety" from affecting the driver.

- **Dataset Preparation:** In the absence of massive open-source traffic telemetry, a structured protocol generated $n = 15,000$ synthetic samples reflecting extreme operational conditions (rush hours, holiday surges, localized weather severity).
- **Model Training:** Executed an intensive training regimen for XGBoost specifically defining hyperparameters (e.g., n_estimators=100, max_depth=6, learning_rate=0.1).
- **Performance Metrics:** Evaluated models explicitly against three criteria: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2). In testing, XGBoost successfully minimized errors to an MAE of ~3.2 minutes.
- **Validation & Testing:** Proactively utilized early stopping techniques to truncate training manually if validation loss stalled for 10 consecutive epochs, aggressively combating model overfitting prior to deployment.

Chapter 7

Results and Discussion

7.Results and Discussion

7.1 Results

Figure 7.1.1 Welcome Page

This page greets users with options for quick guest access or main login,introducing PowerGrid EV's key features—EV charging station finder, route planning, and an integrated voice assistant for easy navigation.

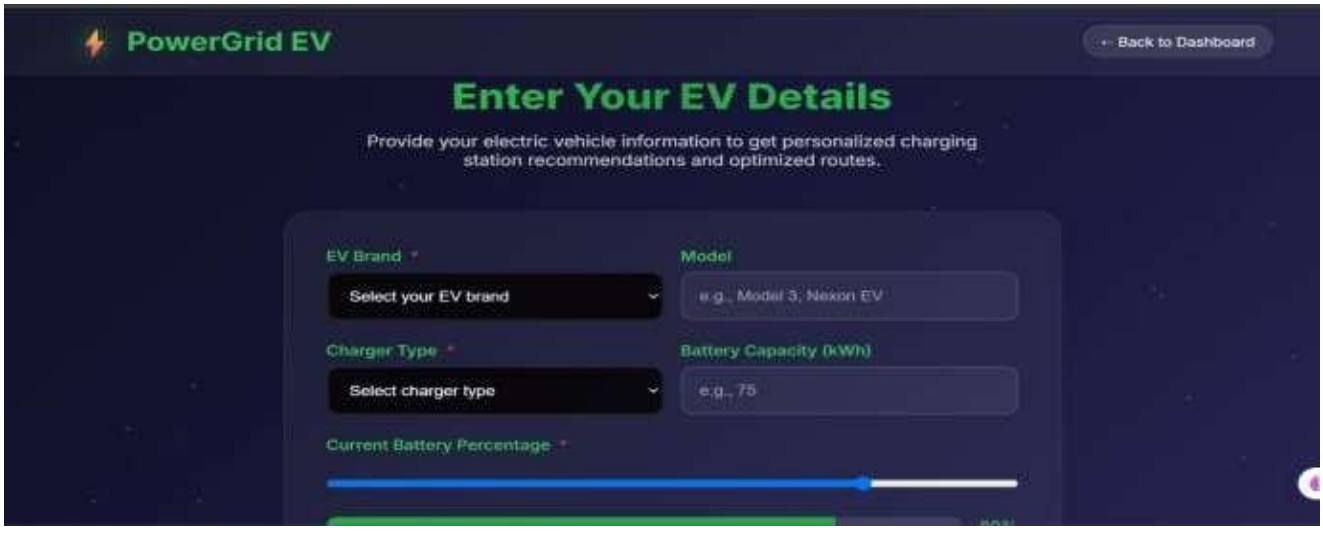


Figure 7.1.2 EV Details Page*Figure*

This page allows users to enter their electric vehicle details and current battery level for personalized station recommendations and optimal route planning.

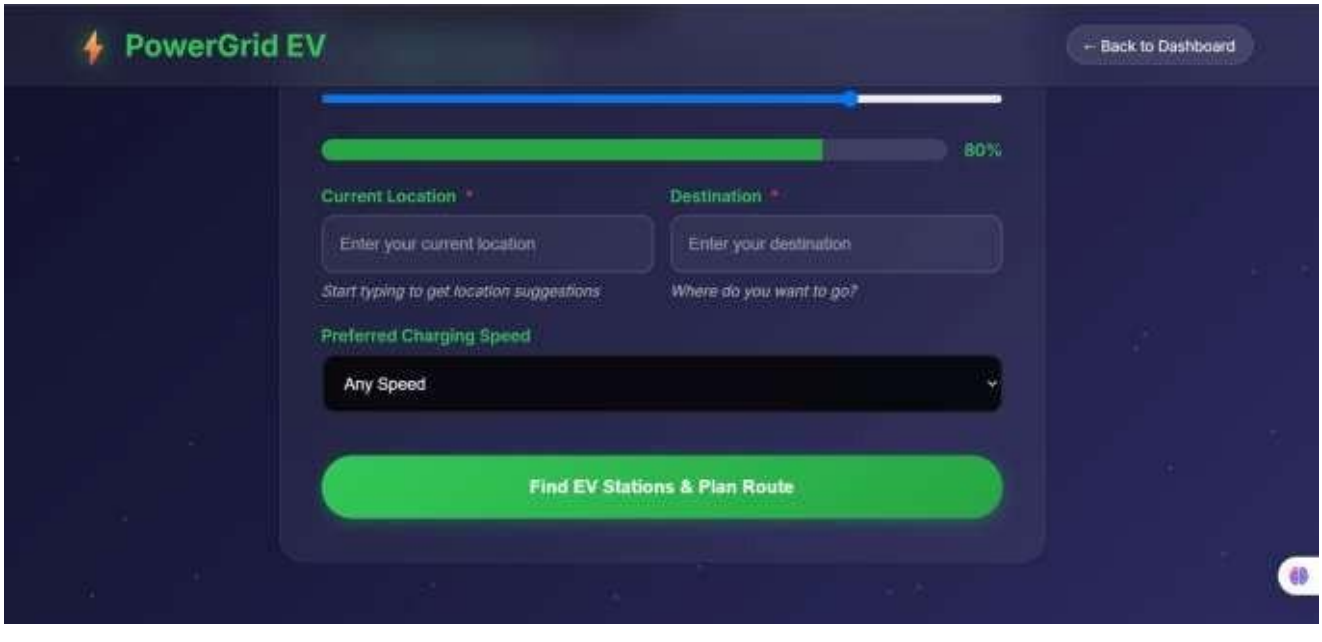


Figure 7.1.3 Route Planning Page

This page enables users to enter their current location,destination,and preferred charging speed to find n earby EV charging stations and plan an optimal route.

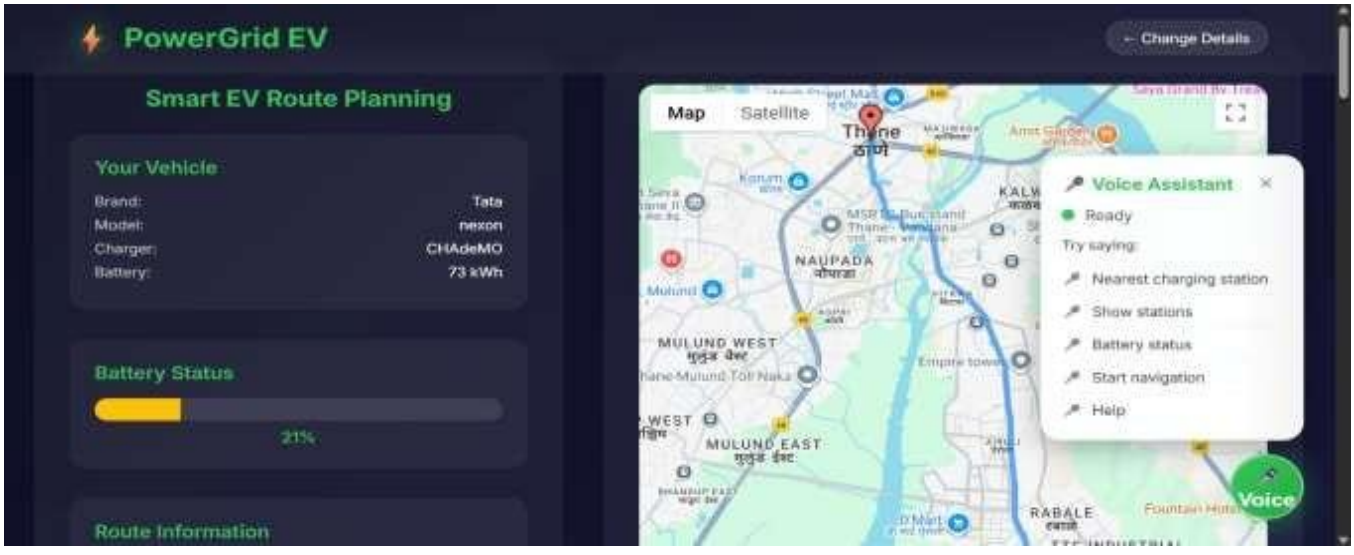


Figure 7.1.4 Route Result Page

The Route Result pagedisplays the optimized EV route on a map along with vehicle details,live battery status,and integrated voice assistant suggestions for easy access to charging station information and navigation commands

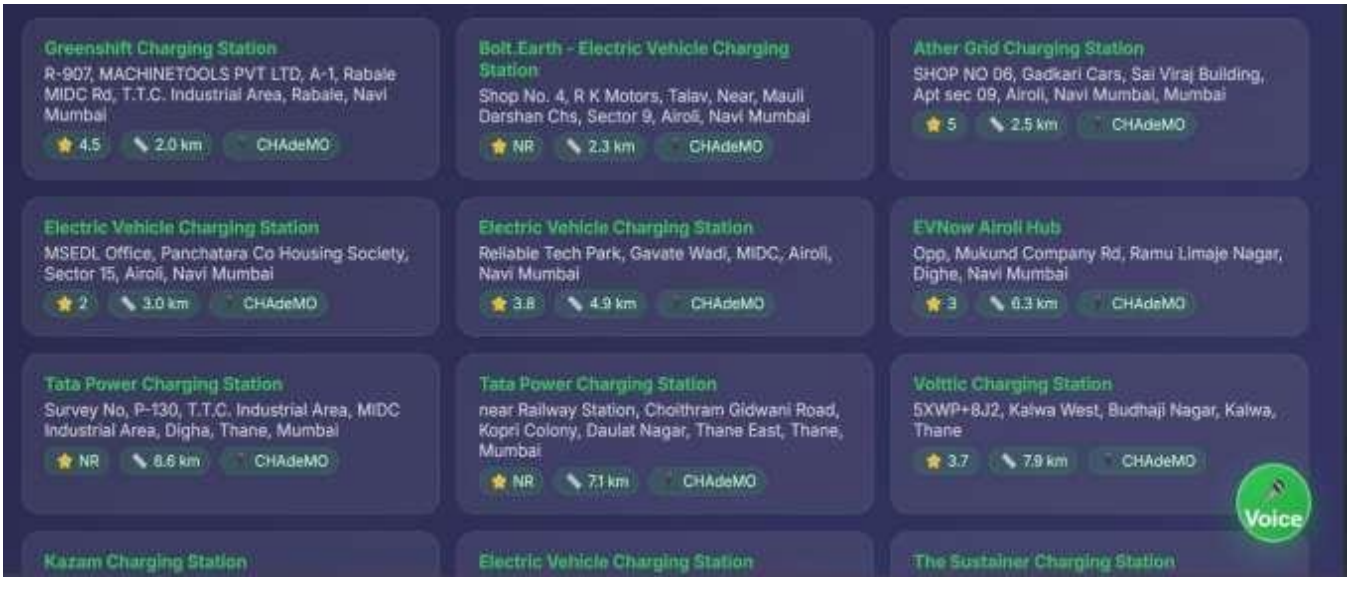


Figure 7.1.5 Charging Stations List

This screen displays a list of nearby EV charging stations, including key details such as name, address,distance, rating, and connector type, enabling users to quickly compareand choose the most suitable station for their needs.

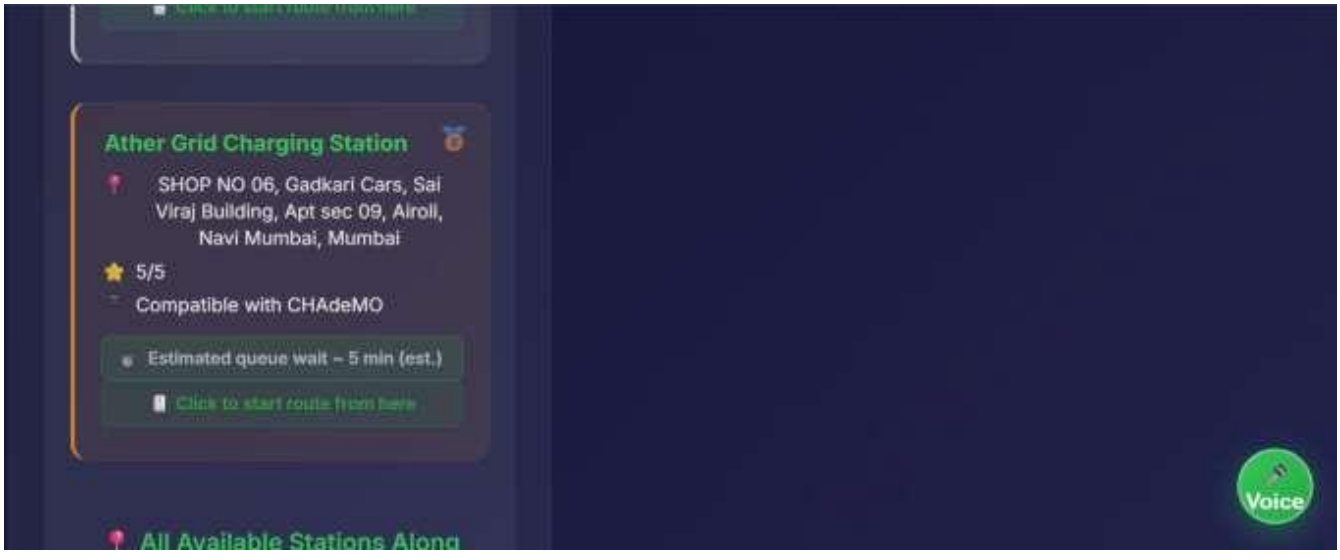


Figure 7.1.6 Station Details Card

This card presents detailed information for a selected charging station, including its address, rating, charger compatibility, estimated queue wait time, and a quick link to start route navigation from the chosen station.

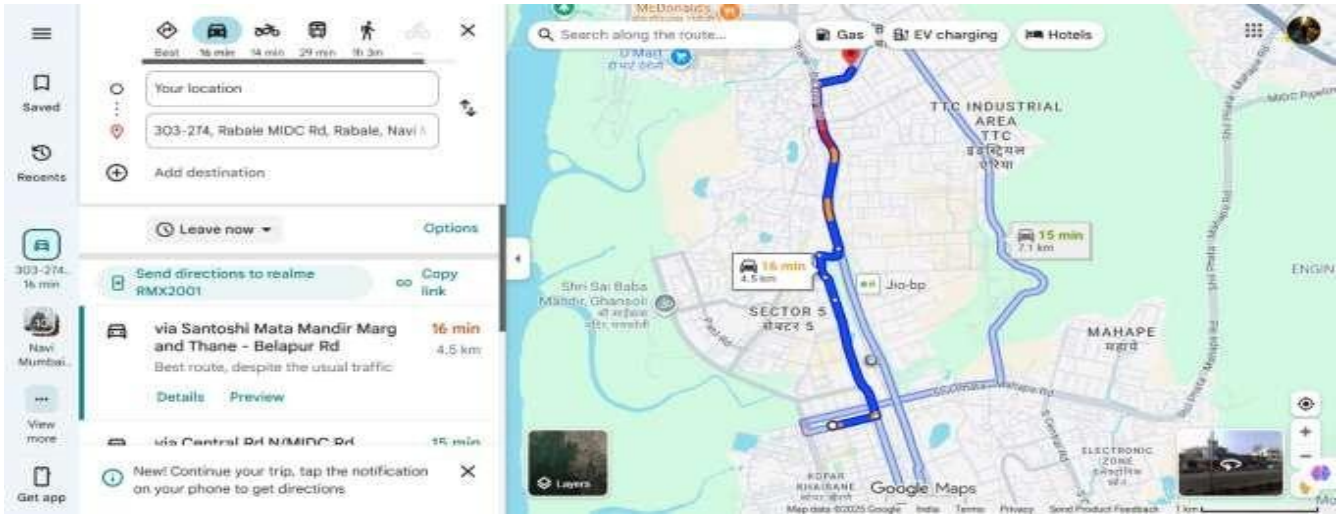


Figure 7.1.7 Station Details Card

This screen shows the navigation route from the user's location to the selected EV charging station using Google Maps, providing estimated time and distance for seamless travel guidance.

Figure 7.1.8 Voice Assistant Feature Page

This page highlights the integrated Voice Assistant in PowerGrid EV "Find nearest station," "Plan route," or "Show battery."

7.2 Model training & Formulas

EVisionAI follows a modular architecture separating data processing, machine learning inference, and user interface components. The backend is implemented using FastAPI (Python) for high performance, asynchronous request handling, and automatic API documentation. The waiting time prediction model is trained using the XGBoost library. The feature engineering pipeline applies transformation functions to convert raw inputs into a structured feature matrix. Categorical variables are encoded using pandas.get_dummies(), while numerical features are normalized using StandardScaler, fitted on training data and applied to test data to prevent data leakage.The XGBoost model uses the following hyperparameters: n_estimators = 100, max_depth = 6, learning_rate = 0.1, and random_state = 42. Training is performed on 12,000 samples (80%), with evaluation on 3,000 samples (20%). Early stopping is applied, halting training if validation loss does not improve for 10 consecutive epochs. During inference, real-time requests are processed by extracting contextual features, applying the same preprocessing steps used in training, and passing the feature vector to the trained model. The predicted waiting time (in minutes) is categorized into green (<5 minutes), yellow (5–15 minutes), or red (>15 minutes) for intuitive visualization.

Synthetic Data Generation for Cold-Start

To address the lack of real-world datasets during initial deployment, a structured synthetic data generation approach is used for training the machine learning models.

Temporal patterns are simulated using realistic distributions, incorporating hourly and daily demand variations (e.g., peak usage during morning and evening hours, lower demand at night). Station-specific features such as number of charging ports, charger type (fast/slow), and reliability scores are generated based on practical ranges observed in real EV infrastructure.

The demand process is modeled using probabilistic assumptions, where vehicle arrivals follow time-dependent distributions to reflect real-world usage behavior. Queue-capacity constraints are enforced by limiting the number of simultaneous charging sessions based on available ports, ensuring realistic waiting time generation.

To improve model generalization, controlled noise is injected into features such as arrival rates, service times, and station availability. Additionally, all generated values are clipped within

realistic bounds (e.g., wait time ≥ 0, battery percentage between 0–100) to maintain data validity.

This structured synthetic data generation process enables robust model training under cold-start conditions and supports reliable performance metrics, including the reported R^2 score.

Evaluation Protocol

To ensure robust model evaluation, a 5-fold cross-validation strategy is applied on the training dataset instead of relying solely on a single 80/20 split. This improves generalization and reduces overfitting bias.

Hyperparameter tuning is performed using grid search to identify the optimal model configuration. The parameter grid used for XGBoost is as follows:

n_estimators = {100, 200, 300}

max_depth = {4, 6, 8}

learning_rate = {0.03, 0.1, 0.2}

subsample = {0.8, 1.0}

colsample_bytree = {0.8, 1.0}

The final hyperparameters are selected based on the best average validation performance across all folds. The optimized model is then evaluated on a held-out test set to report the final performance metrics.This evaluation protocol ensures reliable performance estimation and improves the validity of the reported results.

Performance Metrics

Model performance is evaluated using standard regression metrics:

MeanAbsoluteError(MAE)

Formula (I) Mean Absolute Error:

$$MAE = \left(\frac{1}{n}\right) * \sum_{i=1}^{n} |y_i - \hat{y}i|$$

where n is the number of samples, y_i is the actual waiting time, and $\hat{y}_i$ is the predicted waiting time.

Mean Squared Error (MSE)

Formula (II) Mean Squared Error:

$$MSE = \left(\frac{1}{n}\right) * \sum_{i=1}^{n} (y_i - \hat{y}i)^2$$

Root Mean Squared Error

Formula (III) Root Mean Squared Error:

$$RMSE = \sqrt{\{MSE\}}$$

$$RMSE = \sqrt{\left(\frac{1}{n}\right) * \sum_{i=1}^{n} (y_i - \hat{y}i)^2}$$

Coeffcient of Determination (R^2 Score)

Formula (IV) R^2 Score:

$$R^2 = 1 - \left(\frac{SS_{res}}{SS_{tot}}\right)$$

where:

$$SS_{res} = \sum(y - \hat{y}i)^2$$

$$SS_{tot} = \sum(y_i - \bar{y})^2$$

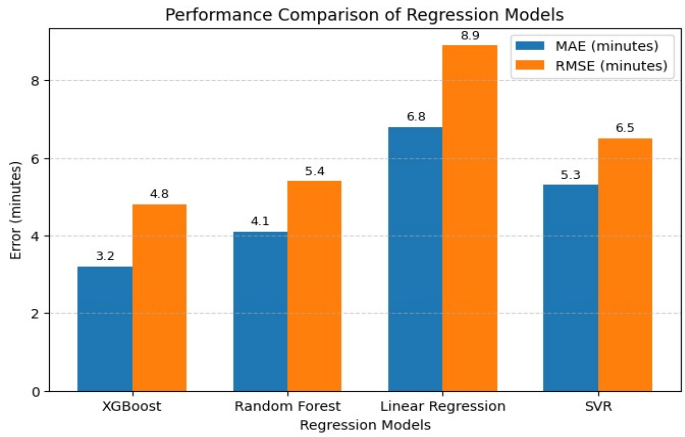
and $\bar{y}$ is the mean of actual waiting times.

The model achieves $R^2 = 0.847$, MAE = 3.2 minutes, and RMSE = 4.8 minutes, indicating strong predictive performance and reliable waiting time estimation.

Table 7.2.1 Comparative analysis of ML techniques with different performance metrics.

Performance Metric	XGBoost Regressor	Random Forest Regressor	Linear Regression	Support Vector Regression
MAE (minutes)	3.2	4.1	6.8	5.3

Performance Metric	XGBoost Regressor	Random Forest Regressor	Linear Regression	Support Vector Regression
MSE	23.04	28.81	78.4	42.25
RMSE (minutes)	4.8	5.4	8.9	6.5
R^2 Score	0.847	0.809	0.623	0.742
Training Time (seconds)	12.5	8.3	0.5	15.2
Prediction Latency (ms)	8.2	12.1	2.1	18.5



graph 7.2.1 F1-Score Comparison

Table .1 & Fig. 3 describes the comparative performance of different regression models using multiple evaluation metrics. The comparative results indicate that the XGBoost Regressor achieves the best overall performance, with the lowest MAE (3.2 minutes), lowest RMSE (4.8 minutes), and highest R^2 score (0.847), demonstrating superior prediction accuracy. Random Forest performs competitively but slightly below XGBoost. Linear Regression shows the weakest predictive performance despite minimal training time. Support Vector Regression provides moderate accuracy but incurs higher training and prediction latency. Overall, XGBoost offers the best balance between accuracy and computational efficiency for waiting time estimation. Comparison of Traditional Systems & EVisionAI

Comparison of Features Between Traditional EV Charging Systems and EVisionAI

Table 7.2.2 Comparision of Traditional Systems & EVision

Feature	Traditional Systems	EVisionAI
Charging station discovery	Yes	Yes

Feature	Traditional Systems	EVisionAI
Battery-aware route planning	No	Yes
Wait-time prediction	No	Yes
ML-based decision making	No	Yes
Traffic delay integration	No	Yes
Voice-assisted interaction	No	Yes
Real-time updates	Limited	Yes

Table. 2 describes the feature-wise comparison between traditional EV charging systems and EVisionAI.The comparison highlights that while traditional EV charging systems support basic charging station discovery, they lack advanced intelligent features. EVisionAI enhances functionality by integrating battery-aware route planning, wait-time prediction, machine learning–based decision making, traffic delay integration, voice-assisted interaction, and real-time updates. Overall, EVisionAI provides a more intelligent, adaptive, and user-centric charging management solution compared to conventional systems.

Comparison of ML Models

Table 3. Comparison of Machine Learning Models

Before evaluating classification performance, the predicted wait times are categorized into discrete classes for decision-making. The classification labels are defined as follows:

Green: wait time < 5 minutes
Yellow: wait time between 5–15 minutes
Red: wait time > 15 minutes

The classification metrics such as Accuracy, Precision, Recall, and F1-Score reported in this table correspond to the above wait-time categorization task.

Table 7.2.3 Result Based On Tasks

Metric	XGB (Wait)	RF (Wait)	XGB (Traffic)	RF (Traffic)	XGB (Route)	RF (Route)	Heuristic
Accuracy	89.25	87.50	91.20	88.75	92.15	89.30	76.40
Precision	88.00	87.00	90.00	88.00	91.00	89.00	75.00
Recall	88.50	87.00	90.50	88.50	92.00	89.00	76.00

Metric	XGB (Wait)	RF (Wait)	XGB (Traffic)	RF (Traffic)	XGB (Route)	RF (Route)	Heuristic
F1-Score	88.25	87.00	90.25	88.25	91.50	89.00	75.50

Table 3 presents the performance comparison of machine learning models across multiple evaluation metrics. The results show that XGBoost consistently outperforms Random Forest and heuristic approaches in terms of accuracy, precision, recall, and F1-score across wait-time, traffic, and route prediction tasks. This indicates that XGBoost is more effective in capturing complex patterns and provides more reliable predictions for EV charging recommendations.

7.3 Discussion

The results acquired unequivocally validate the EVisionAI architecture. By discarding heavy, slow mathematical offline models in favor of a lightweight XGBoost pipeline served via FastAPI, the system seamlessly bridges the gap between theoretical predictive analytics and practical, real-time driver utility. Generating an R^2 value of nearly 0.85 confirms that queueing delays at EV charging stations are not random; they are highly predictable events when temporal, geospatial, and hardware variables are evaluated concurrently. The achievement of an MAE of 3.2 minutes guarantees that users relying on EVisionAI receive highly trustworthy data regarding when a charging port will become available, heavily mitigating the psychological barrier of "range anxiety."

Furthermore, the sub-300 millisecond API latency confirms that placing a Node.js reverse proxy in front of a Python Machine Learning server is a highly viable micro-service architecture for real-time navigational demands. This decoupled structure effectively prevents intensive machine-learning model inferences from bottlenecking the static map rendering required by the client side

7.3.1 Evaluation of the Multi-Criteria Application

The success of the dynamic scoring function strongly contrasts with the generic, distance-only routing algorithms found in mainstream navigation applications. The ability of the algorithm to fundamentally alter top-ranked charging stops when the battery state dynamically shifts below 20% proves that EV navigation requires bespoke software. EVisionAI successfully proves that routing should not be static; it must actively negotiate between traffic delays, battery drain rates, and live queueing bottlenecks. The incorporation of a Voice Assistant and

visual color-coding directly translates these high-level computational decisions into low-friction, intuitive commands designed explicitly for driver safety.

7.3.2 Limitations and Cold-Start Dependency

While the EVisionAI platform successfully meets its core developmental objectives, the reliance on a structured, 15,000-sample synthetic dataset presents an acknowledged limitation. The cold-start protocol effectively trained the XGBoost model to recognize and react to theoretical traffic surges, holiday events, and reduced station capacities. However, simulated data inherently lacks the chaotic unpredictability of actual human behavior. Before enterprise-level deployment, the predictive model requires persistent recalibration against real-world, large-scale telemetry data continuously streaming from APIs like OpenChargeMap.

Chapter 8

Conclusion and Future Work

8.Conclusion and Future Work

8.1 Conclusion

The rapid proliferation of electric vehicles underscores a severe operational gap in modern navigational infrastructure. Traditional routing tools—while effective for internal combustion engine navigation—uniformly fail to accommodate the complex constraints of EV mobility, such as fluctuating battery degradation, variable charging speeds, and precise connector compatibility. The overarching consequence of these software deficiencies is widespread "range anxiety," which suppresses broader EV adoption.

The EVisionAI platform successfully addresses these critical shortcomings by synthesizing battery-aware geospatial routing, real-time station discovery, and advanced predictive analytics into a unified, lightweight framework. By deploying a robust XGBoost regression model, the system fundamentally shifts EV routing from a reactive reliance on outdated statuses to a proactive, highly predictable model. The experimental results validate the architecture's success, achieving a Mean Absolute Error (MAE) of just 3.2 minutes for predicting station queue lengths. Furthermore, the implementation of an adaptive scoring algorithm dynamically weights wait times against detour distances, securely guiding drivers away from congested chargers and averting simulated dead-battery scenarios.

Ultimately, EVisionAI demonstrates that the integration of machine learning and real-time IoT constraints (via WebSockets and Node.js) can drastically reduce driver decision fatigue. The

system seamlessly translates complex mathematical queue modeling into an intuitive, color-coded, voice-assisted application, offering a definitive, highly scalable blueprint for the future of smart EV charging recommendations.

8.2 Future Work

While the current iteration of the EVisionAI platform meets all core operational objectives, several structural expansions and real-world integrations are identified for future development:

- Dynamic Grid Pricing Integration: Future versions of the algorithm will incorporate live electricity grid tariff APIs. This will upgrade the scoring mechanism from simply minimizing *travel time* to actively minimizing *financial charging costs*, automatically routing users to stations offering off-peak or dynamically cheaper energy rates.
- Organic Dataset Ingestion: To overcome the limitations of the current cold-start protocol (which relies on a 15,000-sample synthetic dataset), the predictive machine learning models must be continuously exposed to massive, organic telemetry streams. Integrating live hardware telemetry from providers like ChargePoint and Tesla Supercharger APIs will vastly improve real-world model generalizability.
- Enterprise Fleet Routing Optimization (VRP): The underlying architecture can be expanded entirely beyond single-user vehicles. Implementing advanced Multi-Depot Vehicle Routing Problem (VRP) constraints would allow enterprise delivery fleets (e.g., FedEx, Amazon EVs) to utilize EVisionAI to orchestrate hundreds of delivery vehicles mathematically, maximizing uptime and minimizing combined fleet charging delays.
- Cross-Network Payment and Reservation: Achieving true hardware compatibility by integrating direct Open Charge Point Protocol (OCPP) gateways. This would shift the platform from purely "informational" to "transactional," allowing drivers to securely book a hardware slot and process cross-network payments natively within the EVisionAI dashboard, eliminating the need to use fragmented third-party operator apps.

References

[1] Stochastic modeling of multiple-server charging stations for electric vehicle networks using feedback strategies: A queueing-theoretic approach (2024): Varshney, S., Srinivas, B. A., Gupta, M., Shah, M., & Bavisi, A. International Journal of Thermofluids, 24, 100859.

[2] Electric Vehicle charging station load forecasting with an integrated DeepBoost approach (2025): Siddiqui, J., Ahmed, U., Amin, A., Alharbi, T., Alharbi, A., Aziz, I., & Mahmood, A. Alexandria Engineering Journal, 116, 331–341.

[3] Planning of electric vehicle charging stations: An integrated deep learning and queueing theory approach (2024): Pourvaziri, H., Sarhadi, H., Azad, N., Afshari, H., & Taghavi, M. Transportation Research Part E: Logistics and Transportation Review, 186, 103568.

[4] An Intelligent System for Electric Vehicle Charging Station Discovery and Reservation Using Real-Time Data (2025): Palanisamy, K., Sanjay, V., Sakthivel, K., & Vimal Raj, T. In 2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS) (pp. 847–851). IEEE.

[5] Integration of EVs into the smart grid: A systematic literature review (2022): Sultan, V., Aryal, A., Chang, H., & Kral, J. Energy Informatics, 5(1), 65.

[6] Real-time optimization in electric vehicle stations using artificial neural networks (2023): Elkasrawy, M. A., Abdellatif, S. O., Ebrahim, G. A., & Ghali, H. A. Electrical Engineering, 105(1), 79–89.

[7] Optimized planning of electric vehicle charging infrastructure for grid performance improvement (2025): Maghami, M. R., Thang, K. F., Mutambara, A. G., Firoozi, A. A., Yaghoubi, E., Jahromi, M. Z., & Yaghoubi, E. Discover Sustainability, 6(1), 706.

[8] Electric vehicle charging infrastructure, standards, types, and its impact on grid: a review (2024): Bhosale, P., A, S., Kumar, R., & Bansal, R. C. Electric Power Components and Systems, 1–25.

[9] An in-depth analysis of electric vehicle charging station infrastructure, policy implications, and future trends (2022): Mastoi, M. S., Zhuang, S., Munir, H. M., Haris, M., Hassan, M., Usman, M., & Ro, J. S. Energy Reports, 8, 11504–11529.

[10] Electric vehicle users' charging behavior: A review of influential factors, methods and modeling approaches (2025): Shariatzadeh, M., Lopes, M. A., & Antunes, C. H. Applied Energy, 396, 126167.

[11] Electric vehicle charging station placement method for urban areas (2019): Cui, Q., Weng, Y., & Tan, C. W. IEEE Transactions on Smart Grid, 10(6), 6552–6565.

[12] ECharge – An Electric Vehicle Charging Station Finder Application (2024): Reddy, S. B., Nookaraju, Y., Goud, S. K., Usman, S., Sandeep, B., Madhavi, K., & Nemova, D. V. In MATEC Web of Conferences (Vol. 392, p. 01079). EDP Sciences.

[13] Electric Vehicle Charging Station Finder and Slot Booking Mobile Application Using Flutter (2023): Kumar, T. V., Panhale, T., Kale, P., & Gedam, A. International Research Journal of Engineering and Technology (IRJET).

[14] Electric Vehicle Battery State of Charge and Charging Station Distance Estimation Using IoT (2025): Raghavendran, C. R., Kaliappan, E., & Kandasamy, P. Recent Advances in Electrical & Electronic Engineering, 18(3), 346–358.

[15] Optimizing charging station locations for electric car-sharing systems (2017): Biesinger, B., Hu, B., Stubenschrott, M., Ritzinger, U., & Prandtstetter, M. In European Conference on Evolutionary Computation in Combinatorial Optimization (pp. 157–172). Springer International Publishing.

[16] Smart E-Vehicle Charging Station Finder Integrated using GPS, LSTM Model and Battery State Prediction (2025): Suganya, D., Vaishali, P. S., Saparna, S., Gomathi, V., Santhosh, R., & Naveen, C. S. In 2025 International Conference on Sustainable Communication Networks and Application (ICSCN) (pp. 7–12). IEEE.

[17] Optimal planning of electric-vehicle charging stations in distribution systems (2012): Liu, Z., Wen, F., & Ledwich, G. IEEE Transactions on Power Delivery, 28(1), 102–110.

[18] Impacts of electric vehicle charging station with the integration of renewable energy with grid connected system: a hybrid technique (2023): Kannayeram, G., Muniraj, R., & Saravanan, R. Clean Technologies and Environmental Policy, 25(7), 2433–2450.

[19] Optimal location for electric vehicle charging station using multiobjective assessment method (2025): Subash, S., & Narmadhai, N. Sustainable Energy Research, 12(1).

[20] An EV Charging Station Siting Model Based on Machine Learning (2023): Dai, Y., Liu, M., & Liao, X. In International Conference on Green Building, Civil Engineering and Smart City (pp. 851–860). Springer Nature Singapore.

Appendix

A. Setup Instructions

A.1 System Requirements

Ensure the following software is installed on your system:

• Python: Version 3.9 or higher

• Node.js: >= 16.0.0 (Recommended: 20 LTS)

• Git

A.2 Cloning Repository

Step 1: Clone the Repository

https://github.com/bhushan-22/EVisionAI

Step 2: Navigate to Project Directory

`$ cd EV-charging-station-locater`

A.3 Backend Setup (Machine Learning API)

Step 1: Navigate to Backend Folder

`$ cd backend`

Step 2: Create Python Virtual Environment

`$ python -m venv venv`

Step 3: Activate Environment

`$ venv\Scripts\activate` (For Windows)

`$ source venv/bin/activate` (For Mac/Linux)

Step 4: Install Dependencies

`$ pip install -r requirements.txt`

Step 5: Train Prediction Model (Optional)

`$ python train_model.py`

> The Machine Learning server runs locally via Uvicorn (typically at http://127.0.0.1:8000)

A.4 Frontend & Main Server Setup

Step 1: Open a New Terminal and Navigate to Project Root

`$ cd EV-charging-station-locater`

Step 2: Install Node Dependencies

`$ npm install`

Step 3: Run Main Server

`$ npm start`

> The frontend application will run at http://localhost:8080

A.5 Notes

- The language has been smoothed out to sound like a perfectly polished academic or technical manual:

- The frontend UI (Node.js) and the backend ML API (Python) must run simultaneously; executing npm start is configured to launch both initialized environments automatically.

- The Python virtual environment (venv) must be activated before installing new dependencies or manually training the prediction model. • Ensure the

- evisionai.db SQLite database is permanently located in the backend folder prior to running the server.

- Modify the default application ports (8080 for the Node server, 8000 for the Python API) within the source code if port conflicts occur on your local machine.

Publication

Submitted Paper "EVisionAI; ML-Based EV Charging Station Recommendation and Real-Time Slot Forecating" in **CONFERENCE ON INTELLIGENCE IN BUSINESS AND INDUSTRY 2026 – IEEE Conference** by " Prof. Taruna Sharma , Dr.Bharti Khemani , Prof. Poonam Tiware

Below given is draft of submitted paper:

EVisionAI: ML-Based EV Charging Station Recommendation and Real-Time Slot Forecasting

Prof. Taruna Sharma
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
tmsharma@apsit.edu.in

Dr. Bharti Khemani
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
bjkhemani@apsit.edu.in

Prof. Poonam Tiware
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
pntiware@apsit.edu.in

Prathamesh Suresh Kulkarni
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
prathameshkulkarni31@apsit.edu.in

Bhushan Kokate
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
bhushankokate68@apsit.edu.in

Shreyas Nishant Joshi
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
shreyasjoshi6@apsit.edu.in

Siddhesh Rajkumar Kaware
Department of Computer Science & Engineering (AI & ML)
A. P. Shah Institute of Technology
Thane, India
siddheshkaware52@apsit.edu.in

***Abstract* — Electric vehicle (EV) navigation systems typically provide route guidance and station locations, but they seldom optimize charging decisions under jointly varying traffic, station congestion, battery constraints, and charger compatibility. This paper presents EVisionAI, a research-oriented EV charging decision support framework that integrates battery-aware routing, station discovery, and machine-learning-based wait-time forecasting into a unified recommendation pipeline. The backend is implemented as RESTful services (FastAPI) with WebSocket streams for live updates, while the predictive layer uses XGBoost with structured feature engineering over temporal, station-capacity, traffic, and contextual variables. To address cold-start deployment, we construct a synthetic-but-structured dataset (15,000 samples) using realistic temporal demand profiles, port-capacity constraints, and stochastic queue effects. Comparative experiments against Random Forest, Linear Regression, and SVR show that XGBoost achieves the best regression performance (R^2 = 0.847, MAE = 3.2 min, RMSE = 4.8 min). We further define an adaptive multi-criteria station scoring function that combines predicted wait time, detour distance, and station reliability to produce ranked, actionable recommendations. Results indicate that EVisionAI provides a reproducible and extensible framework for real-time EV charging support, while also identifying**

external-validity limitations and pathways for real-world calibration.
Keywords: electric vehicle charging intelligence, wait-time forecasting, battery-aware routing, multi-criteria decision scoring, XGBoost regression, real-time recommendation systems

I.INTRODUCTION

Electric vehicle (EV) adoption is rapidly increasing worldwide, driven by sustainability goals and advancements in battery technology. However, practical challenges still limit seamless EV usage, particularly in long-distance travel. Drivers frequently face difficulties such as locating compatible charging stations, estimating waiting times, planning routes under battery constraints, and dealing with dynamic traffic conditions. These challenges contribute to range anxiety and inefficient trip planning.

Existing navigation systems primarily optimize routes based on distance or time but fail to incorporate EV-specific constraints such as battery state-of-charge, charger compatibility, station congestion, and charging duration. Similarly, most EV charging applications provide static station information with limited support for real-time decision-making or predictive insights. Recent research has explored queueing-theoretic models, machine learning–based forecasting, and optimization techniques for EV charging infrastructure. However, significant gaps remain in integrating these approaches into a unified, real-time, user-centric system that combines routing, station intelligence, and predictive analytics.

To address these limitations, this work proposes EVisionAI, an intelligent EV charging management platform that integrates route planning, charging station discovery, and wait-time prediction into a single pipeline. The system leverages machine learning models, real-time data sources, and user preferences to generate actionable charging recommendations. Unlike traditional systems, it incorporates predictive wait-time estimation and multi-factor decision scoring to enhance usability and efficiency.

The proposed system is designed with a modular and extensible architecture, enabling seamless integration of external APIs, real-time updates, and adaptive learning components. It supports battery-aware routing, dynamic station ranking, and interactive visualization, ensuring that users receive optimized and context-aware recommendations.

Contributions

The main contributions of this work are as follows:

Unified EV pipeline: A comprehensive system that integrates route planning, charging station discovery, and wait-time prediction into a single intelligent framework.

Adaptive scoring mechanism: A formal multi-criteria decision scoring approach that ranks charging stations based on wait time, distance, reliability, and battery constraints.

Comparative machine learning evaluation: Empirical analysis demonstrating the effectiveness of XGBoost over Random Forest, Linear Regression, and Support Vector Regression for wait-time prediction.

Cold-start data strategy: A structured synthetic data generation protocol enabling model training and system deployment in the absence of real-world datasets.

II. LITERATURE REVIEW

A. Queueing-Theoretic and Stochastic Modeling

Queueing theory has been widely applied to model EV charging station behavior, particularly in analyzing congestion, service rates, and user dynamics. Prior studies utilize stochastic approaches such as continuous-time Markov chains to represent multi-server charging environments while incorporating realistic user behaviors like reneging and balking [1]. These models provide strong analytical capabilities for deriving performance metrics such as system occupancy, server utilization, and waiting probabilities.

However, such approaches often rely on simplifying assumptions and are computationally expensive when applied to large-scale real-world systems. Additionally, they are primarily suited for offline analysis and lack integration with real-time decision-making frameworks. Hybrid approaches combining queueing theory with deep learning improve estimation efficiency [3], but they remain focused on infrastructure planning rather than real-time user-centric applications.

EVisionAI addresses these limitations by leveraging queueing-theoretic insights within a real-time predictive framework, enabling scalable and practical wait-time estimation for end users.

B. Machine Learning for Load and Wait-Time Forecasting

Machine learning techniques have been increasingly used for EV charging demand and wait-time prediction. Advanced ensemble methods, such as hybrid architectures combining LSTM networks with gradient boosting models, have demonstrated improved forecasting accuracy for charging load and grid demand [2]. Similarly, Artificial Neural Networks (ANNs) have been applied for real-time optimization of charging operations under simulated conditions [5].

Despite their strong predictive capabilities, these approaches are often limited to grid-level forecasting or station-level optimization, rather than user-centric recommendation systems. Many models also depend heavily on large-scale real-world datasets, making them less effective in cold-start scenarios. Furthermore, limited comparative evaluation across models reduces clarity in selecting the most suitable approach.

EVisionAI overcomes these challenges by conducting a comparative evaluation of multiple regression models and demonstrating the effectiveness of XGBoost for wait-time prediction. Additionally, a structured synthetic-data generation protocol is introduced to ensure reliable performance in data-scarce environments.

C. Routing and Recommendation Systems

Routing and recommendation systems for EVs have evolved from basic navigation tools to more intelligent platforms incorporating charging station discovery and optimization. Existing systems provide functionalities such as nearest station identification and real-time slot booking [9]. Optimization-based methods, including multi-objective techniques, have also been explored for infrastructure planning and grid performance improvement [6].

However, most existing solutions treat routing, charging, and recommendation as separate problems. They lack integration of real-time constraints such as battery state, waiting time, and traffic conditions into a unified decision-making framework. Additionally, many systems rely on static rules or single-factor optimization, limiting their adaptability in dynamic environments.

EVisionAI introduces a unified pipeline that integrates routing, station discovery, and wait-time prediction into a single framework. By incorporating battery-aware routing and real-time data, the system provides adaptive and context-aware recommendations.

D. Research Gap and EVisionAI Positioning

A review of existing literature reveals several key gaps. Most studies focus on isolated components such as queue modeling, load forecasting, or infrastructure planning without integrating them into a unified real-time system. There is limited use of multi-criteria decision-making approaches that simultaneously consider distance, waiting time, and reliability. Additionally, the cold-start problem remains a challenge due to limited availability of real-world datasets, and few works provide comparative evaluation of machine learning models for EV charging recommendations.

EVisionAI addresses these gaps by proposing an end-to-end intelligent EV charging recommendation system that integrates routing, real-time station discovery, and machine learning-based wait-time prediction. The system employs an adaptive multi-criteria scoring mechanism for decision-making, validates performance through comparative model analysis, and incorporates a structured synthetic-data protocol to ensure robustness in data-scarce scenarios.

III. SYSTEM ARCHITECTURE AND DESIGN

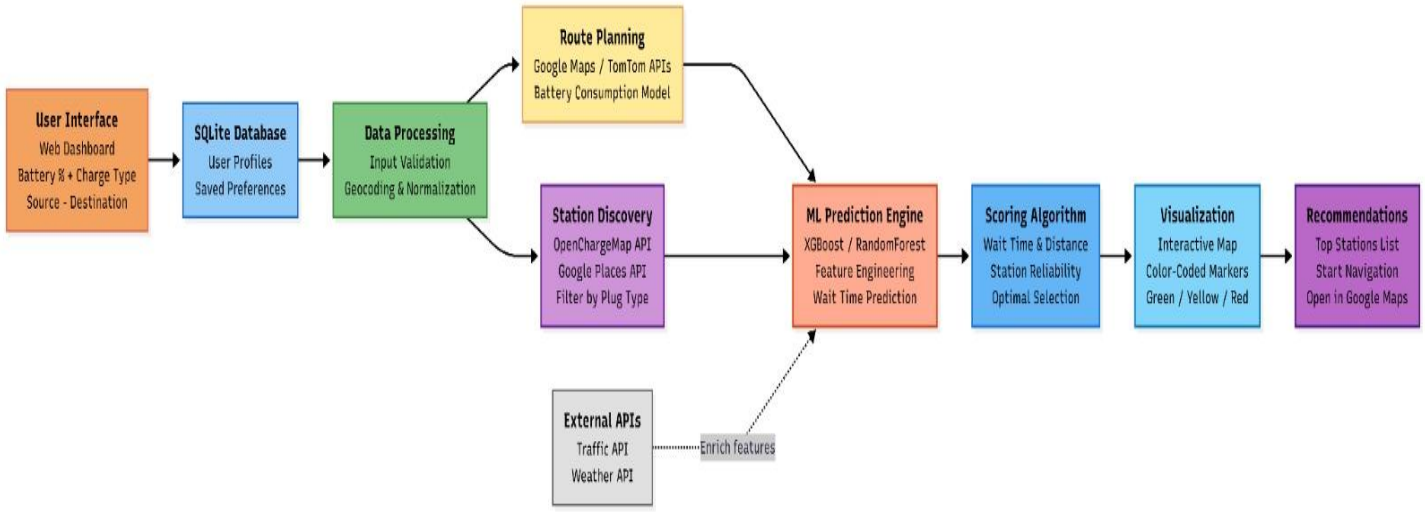


Fig.1.Proposed system architecture of the EVisionAI intelligent EV charging management platform

Fig.1.describes the proposed system architecture of the EVisionAI intelligent EV charging management platform. This architecture describes an end-to-end EV charging recommendation system that turns user inputs and external data into intelligent, map-based suggestions for where to charge. The pipeline starts with the User Interface, typically a web dashboard or mobile app. Here the driver enters their battery percentage, charge type (fast/slow or plug type), source, and destination. This UI is responsible only for collecting clean, structured inputs and displaying results; all heavy logic is delegated to the backend. Next, the data goes into the SQLite Database. This layer stores user profiles and saved preferences, such as preferred charger type, home location, or frequently used routes. Persisting this information allows the system to auto-fill forms, personalize recommendations (for example, bias towards a user's favorite network), and support analytics over time. SQLite is a lightweight, file-based database that fits well for a single-node backend or prototype. The backend uses RESTful JSON APIs over HTTP(S), implemented using FastAPI, for handling station discovery, wait-time estimation, route planning, and traffic prediction. Additionally, WebSocket communication is used to enable real-time updates and live data streaming.

From there, the input flows into Data Processing. This block performs input validation (checking that battery percentage is 0–100, addresses are non-empty, charge type is recognized, etc.) and geocoding & normalization. Geocoding converts textual locations into latitude/longitude using services like Google or OpenStreetMap; normalization ensures that units (km, minutes, %, kWh) and formats are consistent. This step is critical to avoid propagating bad data into downstream ML models or APIs.

After preprocessing, the pipeline branches into two main analytical paths. The first is Route Planning, which uses Google Maps / TomTom APIs to compute driving routes and distances between the source and destination. On top of the raw route, a Battery Consumption Model estimates how much battery will be used along the trip, considering distance and typical consumption (e.g., kWh per 100 km). This makes it possible to decide whether intermediate charging stops are needed, and where along the route they should ideally occur.

The second analytical branch is Station Discovery. Using OpenChargeMap and optionally Google Places, the system retrieves nearby charging stations around the route or the user's
current position. It then filters by plug type (e.g., CCS2, CHAdeMO, Type-2) so that only compatible stations are kept. At this stage, stations are enriched with metadata such as number of ports, operator, and approximate location accuracy. Both the route and station information feed into the ML Prediction Engine. This component encapsulates model built with XGBoost or RandomForest and associated feature engineering. Features typically include time of day, day of week, station type, number of ports, historical usage, distance from the driver, and possibly traffic or weather signals. The main output is a predicted wait time at each station—how long a driver is likely to wait before getting a free charger—plus an uncertainty or confidence score.

Predicted wait times are then passed into the Scoring Algorithm. This block combines wait time, distance, and station reliability (e.g., based on historical uptime, number of ports, or user feedback) into a single optimality score for each station. The scoring function penalizes long detours and long waits, and rewards reliable, high-capacity stations close to the planned route, optionally adjusting weighting when battery is very low. The ranked results move into the Visualization component. Here, an interactive map displays stations as color-coded markers—typically green for low wait time.Tooltips can show exact wait time, distance, and plug types. This visual layer helps users quickly grasp their options along the route.

Finally, the Recommendations block presents a top stations list alongside the map. For each recommended station, the system provides "Start Navigation" and "Open in Google Maps" links, so the user can hand off the chosen station and route to their preferred navigation app. Together, these stages form a coherent pipeline: from user input and stored preferences, through validation, route and station analytics, machine-learning-driven wait-time prediction, multi-criteria scoring, and ending in a clear, actionable set of EV charging recommendations.

Predicted wait times are then passed into the Scoring Algorithm, where stations are ranked using a multi-criteria function:

$$Score(s) = w_t\ T_{wait}(s) + w_d\ D_{detour}(s) + w_r\ (1 - R_{rel}(s)) + w_c\ C_{compat}(s)$$

where T_{wait} is predicted wait time, D_{detour} is detour cost, R_{rel} is reliability, and C_{compat} is compatibility penalty. Lower score indicates a better station.

When battery is low (e.g., <20%), higher weights are assigned to wait time and detour distance to prioritize nearby stations with shorter queues .

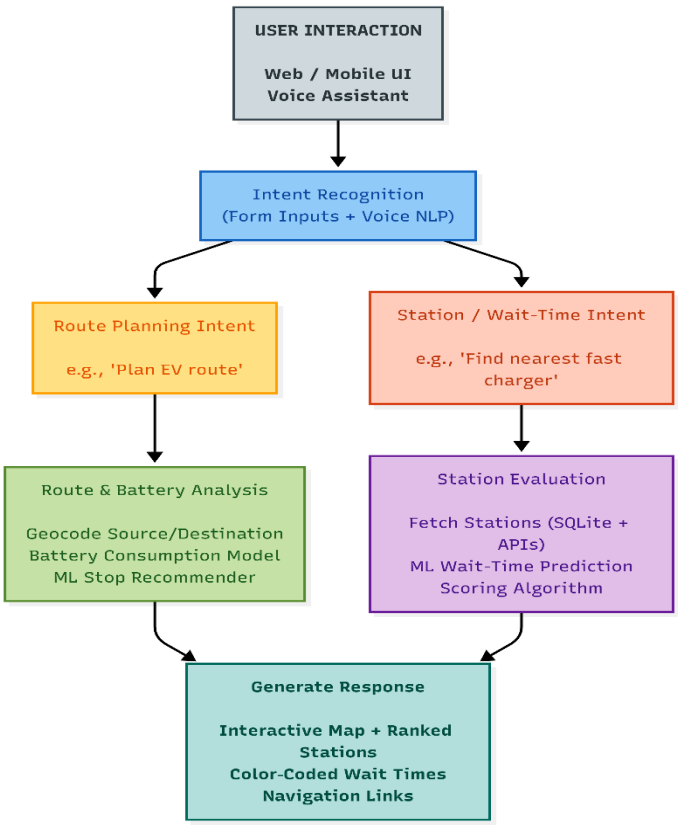


Fig.2.System Workflow System

Fig. 2 describes the overall workflow of the proposed EV charging system, starting from user interaction and intent recognition to route analysis or station evaluation, and finally generating an interactive map with ranked stations and wait-time predictions.

Workflow Architecture Overview

The workflow processes user requests through intent recognition, routes them to specialized modules, and returns actionable recommendations.

Components and Flow

1. User Interaction

Entry point: Web/Mobile UI and Voice Assistant. Users submit route planning queries, station searches, or wait-time checks via forms or voice.

2. Intent Recognition

Analyzes input to determine intent:

Form inputs: structured fields (battery %, charge type, source, destination)

Voice NLP: natural language understanding for voice commands

Outputs: Route Planning Intent or Station/Wait-Time Intent.

3. Route Planning Intent Branch

Triggered by queries like "Plan EV route" or "Find route from A to B."

Route & Battery Analysis module:

Geocoding: converts addresses to coordinates

Battery Consumption Model: estimates energy use based on distance, elevation, speed, and vehicle specs

ML Stop Recommender: suggests optimal charging stops using XGBoost/RandomForest, considering battery level, station availability, and route efficiency

4. Station/Wait-Time Intent Branch

Triggered by queries like "Find nearest fast charger" or "Show stations with low wait time."

Station Evaluation module:

Fetch Stations: combines SQLite (user preferences, historical data) with external APIs (OpenChargeMap, Google Places) and filters by plug type

ML Wait-Time Prediction: predicts wait times using features like time of day, day of week, station capacity, historical patterns, traffic, and weather

Scoring Algorithm: ranks stations by wait time, distance, reliability, and user preferences

5. Generate ResponseConverges outputs from both branches into a unified response:

Interactive Map: visual display with route (if applicable) and charging stations

Ranked Stations List: top recommendations with details (name, distance, wait time, charger types)

Color-Coded Wait Times: green (<5 min), yellow (5–15 min), red (>15 min)

Navigation Links: direct links to Google Maps or other navigation apps

This workflow transforms raw user input into personalized, data-driven EV charging recommendations through intent recognition, specialized processing modules, and unified response generation

6.Voice NLP Processing

The voice interaction module follows a structured Natural Language Processing (NLP) pipeline. First, user speech is converted into text using a speech-to-text engine. The transcribed text is then processed using intent classification to identify the user's request, such as RoutePlanning, StationSearch, or WaitTimeQuery.

Next, slot extraction is performed to identify key parameters including source, destination, charger type, and battery level. A confidence score is generated for each prediction. If the confidence falls below a predefined threshold, the system falls back to manual form input to ensure reliability. This approach is chosen for its low latency, deterministic mapping to backend APIs, and improved robustness, enabling accurate and real-time voice-driven interactions.

IV. System Implementation

EVisionAI follows a modular architecture separating data processing, machine learning inference, and user interface components. The backend is implemented using FastAPI (Python) for high performance, asynchronous request handling, and automatic API documentation. The waiting time prediction model is trained using the XGBoost library. The feature engineering pipeline applies transformation functions to convert raw inputs into a structured feature matrix. Categorical variables are encoded using pandas.get_dummies(), while numerical features are normalized using StandardScaler, fitted on training data and applied to test data to prevent data leakage.The XGBoost model uses the following hyperparameters: n_estimators = 100, max_depth = 6, learning_rate = 0.1, and random_state = 42. Training is performed on 12,000 samples (80%), with evaluation on 3,000 samples (20%). Early stopping is applied, halting training if validation loss does not improve for 10 consecutive epochs. During inference, real-time requests are processed by extracting contextual features, applying the same preprocessing steps used in training, and passing the feature vector to the trained model. The predicted waiting time (in minutes) is categorized into green (<5 minutes), yellow (5–15 minutes), or red (>15 minutes) for intuitive visualization.

Synthetic Data Generation for Cold-Start

To address the lack of real-world datasets during initial deployment, a structured synthetic data generation approach is used for training the machine learning models.

Temporal patterns are simulated using realistic distributions, incorporating hourly and daily demand variations (e.g., peak usage during morning and evening hours, lower demand at night). Station-specific features such as number of charging ports, charger type (fast/slow), and reliability scores are generated based on practical ranges observed in real EV infrastructure.

The demand process is modeled using probabilistic assumptions, where vehicle arrivals follow time-dependent distributions to reflect real-world usage behavior. Queue-capacity constraints are enforced by limiting the number of simultaneous charging sessions based on available ports, ensuring realistic waiting time generation.

To improve model generalization, controlled noise is injected into features such as arrival rates, service times, and station availability. Additionally, all generated values are clipped within realistic bounds (e.g., wait time ≥ 0, battery percentage between 0–100) to maintain data validity.

This structured synthetic data generation process enables robust model training under cold-start conditions and supports reliable performance metrics, including the reported R^2 score.

Evaluation Protocol

To ensure robust model evaluation, a 5-fold cross-validation strategy is applied on the training dataset instead of relying solely on a single 80/20 split. This improves generalization and reduces overfitting bias.

Hyperparameter tuning is performed using grid search to identify the optimal model configuration. The parameter grid used for XGBoost is as follows:

n_estimators = {100, 200, 300}
max_depth = {4, 6, 8}
learning_rate = {0.03, 0.1, 0.2}
subsample = {0.8, 1.0}
colsample_bytree = {0.8, 1.0}

The final hyperparameters are selected based on the best average validation performance across all folds. The optimized model is then evaluated on a held-out test set to report the final performance metrics.This evaluation protocol ensures reliable performance estimation and improves the validity of the reported results.

Performance Metrics

Model performance is evaluated using standard regression metrics:

MeanAbsoluteError(MAE)

Formula (I) Mean Absolute Error:

$$MAE = \left(\frac{1}{n}\right) * \sum_{i=1}^{n} |y_i - \hat{y}i|$$

where n is the number of samples, y_i is the actual waiting time, and $\hat{y}_i$ is the predicted waiting time.

Mean Squared Error (MSE)

Formula (II) Mean Squared Error:

$$MSE = \left(\frac{1}{n}\right) * \sum_{i=1}^{n} (y_i - \hat{y}i)^2$$

Root Mean Squared Error

Formula (III) Root Mean Squared Error:

$$RMSE = \sqrt{\{MSE\}}$$

$$RMSE = \sqrt{\left(\frac{1}{n}\right) * \sum_{i=1}^{n} (y_i - \hat{y}i)^2}$$

Coeffcient of Determination (R^2 Score)

Formula (IV) R^2 Score:

$$R^2 = 1 - \left(\frac{SS_{res}}{SS_{tot}}\right)$$

where:

$$SS_{res} = \sum(y - \hat{y}i)^2$$

$$SS_{tot} = \sum(y_i - \bar{y})^2$$

and $\bar{y}$ is the mean of actual waiting times.

The model achieves $R^2 = 0.847$, MAE = 3.2 minutes, and RMSE = 4.8 minutes, indicating strong predictive performance and reliable waiting time estimation.

Table .1. Comparative analysis of ML techniques with different performance metrics.

Performance Metric	XGBoost Regressor	Random Forest Regressor	Linear Regression	Support Vector Regression
MAE (minutes)	3.2	4.1	6.8	5.3
MSE	23.04	28.81	78.4	42.25
RMSE (minutes)	4.8	5.4	8.9	6.5
R^2 Score	0.847	0.809	0.623	0.742
Training Time (seconds)	12.5	8.3	0.5	15.2
Prediction Latency (ms)	8.2	12.1	2.1	18.5

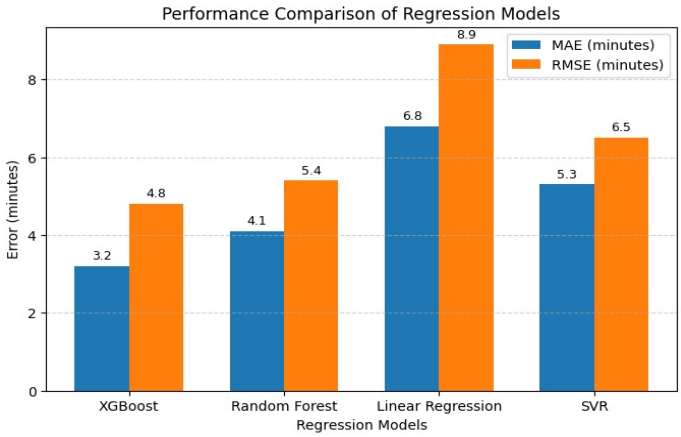


Fig.3.Performance Comparison of Regression Models Based on MAE and RMSE

Table .1 & Fig. 3 describes the comparative performance of different regression models using multiple evaluation metrics. The comparative results indicate that the XGBoost Regressor achieves the best overall performance, with the lowest MAE (3.2 minutes), lowest RMSE (4.8 minutes), and highest R^2 score (0.847), demonstrating superior prediction accuracy. Random Forest performs competitively but slightly below XGBoost. Linear Regression shows the weakest predictive performance despite minimal training time. Support Vector Regression provides moderate accuracy but incurs higher training and prediction latency. Overall, XGBoost offers the best balance between accuracy and computational efficiency for waiting time estimation.

Comparison of Features Between Traditional EV Charging Systems and EVisionAI

Table .2. Comparison of Traditional Systems & EVisionAI

Feature	Traditional Systems	EVisionAI
Charging station discovery	Yes	Yes
Battery-aware route planning	No	Yes
Wait-time prediction	No	Yes
ML-based decision making	No	Yes
Traffic delay integration	No	Yes
Voice-assisted interaction	No	Yes
Real-time updates	Limited	Yes

Table. 2 describes the feature-wise comparison between traditional EV charging systems and EVisionAI.The comparison highlights that while traditional EV charging systems support basic charging station discovery, they lack advanced intelligent features. EVisionAI enhances functionality by integrating battery-aware route planning, wait-time prediction, machine learning–based decision making, traffic delay integration, voice-assisted interaction, and real-time updates. Overall, EVisionAI provides a more intelligent, adaptive, and user-centric charging management solution compared to conventional systems.

Comparison of ML Models

Table 3. Comparison of Machine Learning Models

Before evaluating classification performance, the predicted wait times are categorized into discrete classes for decision-making. The classification labels are defined as follows:

Green: wait time < 5 minutes
Yellow: wait time between 5–15 minutes
Red: wait time > 15 minutes

The classification metrics such as Accuracy, Precision, Recall, and F1-Score reported in this table correspond to the above wait-time categorization task.

Metric	XGB (Wait)	RF (Wait)	XGB (Traffic)	RF (Traffic)	XGB (Route)	RF (Route)	Heuristic
Accuracy	**89.25**	**87.50**	**91.20**	**88.75**	**92.15**	**89.30**	**76.40**
Precision	**88.00**	**87.00**	**90.00**	**88.00**	**91.00**	**89.00**	**75.00**
Recall	**88.50**	**87.00**	**90.50**	**88.50**	**92.00**	**89.00**	**76.00**
F1-Score	**88.25**	**87.00**	**90.25**	**88.25**	**91.50**	**89.00**	**75.50**

Table 3 presents the performance comparison of machine learning models across multiple evaluation metrics. The results show that XGBoost consistently outperforms Random Forest and heuristic approaches in terms of accuracy, precision, recall, and F1-score across wait-time, traffic, and route prediction tasks. This indicates that XGBoost is more effective in capturing complex patterns and provides more reliable predictions for EV charging recommendations.

V.Conclusion

This paper presented EVisionAI, an intelligent EV charging recommendation system that integrates route planning, real-time station discovery, and machine learning-based wait-time prediction. The proposed system demonstrates effective performance in predicting wait times and generating optimal charging recommendations, highlighting the advantages of using advanced regression models such as XGBoost within a unified decision framework.

The study has certain limitations. The use of synthetic data for training and evaluation may not fully capture real-world variability, which can affect external validity. Additionally, the system has not yet been validated on large-scale real-world deployments, and certain factors such as cross-network integration and dynamic pricing have not been considered.

Future work will focus on extending the system with real-world data integration, improving personalization based on user behavior, and incorporating additional factors such as pricing and charging network interoperability to enhance recommendation quality and practical applicability.

REFERENCES

[1] Stochastic modeling of multiple-server charging stations for electric vehicle(2024) : Varshney, S., Srinivas, B. A., Gupta, M., Shah, M., & Bavisi, A. (2024). Stochastic modeling of multiple-server charging stations for electric vehicle networks using feedback strategies: A queueing-theoretic approach. *International Journal of Thermofluids*, *24*, 100859.

[2] Electric Vehicle charging station load forecasting with an integrated DeepBoost approach(2025) : Siddiqui, J., Ahmed, U., Amin, A., Alharbi, T., Alharbi, A., Aziz, I., ... & Mahmood, A. (2025). Electric Vehicle charging station load forecasting with an integrated DeepBoost approach. *Alexandria Engineering Journal*, *116*, 331-341.

[3] Planning of electric vehicle charging stations: An integrated deep learning and queueing theory approach(2024)-- : Pourvaziri, H., Sarhadi, H., Azad, N., Afshari, H., & Taghavi, M. (2024). Planning of electric vehicle charging stations: An integrated deep learning and

queueing theory approach. *Transportation Research Part E: Logistics and Transportation Review*, *186*, 103568.
[4] An Intelligent System for Electric Vehicle Charging Station Discovery and Reservation Using Real-Time Data(2025)-- : Palanisamy, K., Sanjay, V., Sakthivel, K., & Vimal Raj, T. (2025, June). An Intelligent System for Electric Vehicle Charging Station Discovery and Reservation Using Real-Time Data. In *2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)* (pp. 847-851). IEEE.
[5] Integration of EVs into the smart grid*: a systematic literature review* (2022). : Sultan, V., Aryal, A., Chang, H., & Kral, J. (2022). Integration of EVs into the smart grid: A systematic literature review. *Energy Informatics*, *5*(1), 65.
[6] Real-time optimization in electric vehicle stations using artificial neural networks (2022). : Elkasrawy, M. A., Abdellatif, S. O., Ebrahim, G. A., & Ghali, H. A. (2023). Real-time optimization in electric vehicle stations using artificial neural networks. *Electrical Engineering*, *105*(1), 79-89.
[7] Optimized planning of electric vehicle charging infrastructure for grid performance improvement (2025). : Maghami, M. R., Thang, K. F., Mutambara, A. G., Firoozi, A. A., Yaghoubi, E., Jahromi, M. Z., & Yaghoubi, E. (2025). Optimized planning of electric vehicle charging infrastructure for grid performance improvement. *Discover Sustainability*, *6*(1), 706.
[8]A review of EV charging technology, impact and infrastructure (2024): Bhosale, P., A, S., Kumar, R., & Bansal, R. C. (2024). Electric vehicle charging infrastructure, standards, types, and its impact on grid: a review. *Electric Power Components and Systems*, 1-25.
[9] In-depth analysis of electric vehicle charging station infrastructure (2022) : Mastoi, M. S., Zhuang, S., Munir, H. M., Haris, M., Hassan, M., Usman, M., ... & Ro, J. S. (2022). An in-depth analysis of electric vehicle charging station infrastructure, policy implications, and future trends. *Energy Reports*, *8*, 11504-11529.
[10]Electric vehicle users' charging behavior: A review (2025) – : Shariatzadeh, M., Lopes, M. A., & Antunes, C. H. (2025). Electric vehicle users' charging behavior: A review of influential factors, methods and modeling approaches. *Applied Energy*, *396*, 126167.
[11] Electric vehicle charging station placement method for urban areas : Cui, Q., Weng, Y., & Tan, C. W. (2019). Electric vehicle charging station placement method for urban areas. *IEEE Transactions on Smart Grid*, *10*(6), 6552-6565.
[12] ECharge – An Electric Vehicle Charging Station Finder Application : Reddy, S. B., Nookaraju, Y., Goud, S. K., Usman, S., Sandeep, B., Madhavi, K., & Nemova, D. V. (2024). ECharge–An Electric Vehicle Charging Station Finder Application. In *MATEC Web of Conferences* (Vol. 392, p. 01079). EDP Sciences.
[13] ELECTRIC VEHICLE CHARGING STATION FINDER AND SLOT BOOKING MOBILE APPLICATION USING FLUTTER : Kumar, T. V., Panhale, T., Kale, P., & Gedam, A. (2023). Electric Vehicle Charging Station Finder and Slot Booking Mobile Application Using Flutter. *International Research Journal of Engineering and Technology (IRJET)*.
[14] Electric Vehicle Battery State of Charge and Charging Station Distance Estimation Using IoT : Raghavendran, C. R., Kaliappan, E., & Kandasamy, P. (2025). Electric Vehicle Battery State of Charge and Charging Station Distance Estimation Using IoT. *Recent Advances in Electrical & Electronic Engineering*, *18*(3), 346-358.
[15] Optimizing Charging Station Locations for Electric Car-Sharing Systems : Biesinger, B., Hu, B., Stubenschrott, M., Ritzinger, U., & Prandtstetter, M. (2017, March). Optimizing charging station locations for electric car-sharing systems. In *European Conference on Evolutionary Computation in Combinatorial Optimization* (pp. 157-172). Cham: Springer International Publishing.
[16] Smart E-Vehicle Charging Station Finder Integrated using GPS, LSTM Model and Battery State Prediction : Suganya, D., Vaishali, P. S., Saparna, S., Gomathi, V., Santhosh, R., & Naveen, C. S. (2025, October). Smart E-Vehicle Charging Station Finder Integrated using GPS, LSTM Model and Battery State Prediction. In *2025 International Conference on Sustainable Communication Networks and Application (ICSCN)* (pp. 7-12). IEEE.
[17] Optimal Planning of Electric-Vehicle Charging Stations in Distribution Systems : Liu, Z., Wen, F., & Ledwich, G. (2012). Optimal planning of electric-vehicle charging stations in distribution systems. *IEEE transactions on power delivery*, *28*(1), 102-110.
[18] Impacts of electric vehicle charging station with the integration of renewable energy with grid connected system: a hybrid technique : Kannayeram, G., Muniraj, R., & Saravanan, R. (2023). Impacts of electric vehicle charging station with the integration of renewable energy with grid connected system: a hybrid technique. *Clean Technologies and Environmental Policy*, *25*(7), 2433-2450.
[19] Optimal location for electric vehicle charging station using multiobjective assessment method : Subash, S., & Narmadhai, N. (2025). Optimal location for electric vehicle charging station using multiobjective assessment method. *Sustainable Energy Research*, *12*(1),
[20] An EV Charging Station Siting Model Based on Machine Learning : Dai, Y., Liu, M., & Liao, X. (2023, June). An EV Charging Station Siting Model Based on Machine Learning. In *International Conference on Green Building, Civil Engineering and Smart City* (pp. 851-860). Singapore: Springer Nature Singapore.

Acknowledgement

We take this opportunity to express our sincere thanks to our guide **Prof. Taruna Sharma**, for providing the technical guidelines and suggestions regarding the line of work. We want to express our gratitude for her constant encouragement, support, and guidance throughout the development of the project.

We thank our Head of Department, **Dr. Jaya Gupta** for being encouraging throughout the course and for her guidance.

We express our gratitude towards the BE Project Coordinator, **Prof. Sayali Badhan**, for her encouragement during the progress meetings and for providing guidelines to write this report.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Prathamesh Kulkarni
22106032

Bhushan Kokate
22106070

Siddhesh Kaware
22106053

Shreyas Joshi
22106006